Jakarta Persistence Applied 온라인 부록

SQL Server WideWorldImporters Persistence

권진

Version 1.0: Jakarta Persistence 3.2 판

Table of Contents

SQL Server WideWorldImporters Persistence 1
sqlserver-samples-persistence-wideworldimporters 2
Wide World Importers 3
Attribute Mappings. 30
Exercises 40

SQL Server WideWorldImporters Persistence

이 장에서는 SQL Server 의 예제 데이터베이스 가운데 WideWorldImporters 를 매핑한다. 신기한 소품을 들여와 도매로 파는 회사의 판매, 구매, 창고 업무를 담은 데이터베이스다.

데이터베이스를 시작하는 방법과 영속성 유닛, 매핑된 슈퍼클래스와 엔티티를 나누는 규칙, SQL Server 의 형은 [sql-server-sample-databases-persistence] 장에서 보았다. 이 장은 그 규칙을 되풀이하지 않는다. 테이블을 하나씩 살펴보지도 않는다. 먼저 WideWorldImporters 를 매핑한 `sqlserver-samples-persistence-wideworldimporters` 모듈의 구성을 살펴본다. 다음으로 WideWorldImporters 의 스키마와 테이블이 어떻게 이어지는지 보고, 이 데이터베이스가 매핑에 더하는 것을 주제마다 하나씩 살펴본다. 시스템 버전 관리 임시 테이블, 시퀀스, 모든 테이블이 참조하는 `Application.People`, 그리고 `Website` 스키마의 뷰다. 그다음 컬럼을 속성에 어떻게 매핑했는지 요약하고, `geography` 와 바이너리 컬럼, `nvarchar(max)` 에 담은 JSON 과 그 JSON 에서 계산하는 컬럼을 살펴본다. 마지막으로 연습문제를 푼다.

sqlserver-samples-persistence-wideworldimporters

`sqlserver-samples-persistence-wideworldimporters` 는 WideWorldImporters 를 Jakarta Persistence 로 매핑한 모듈이다. WideWorldImporters 가 어떤 데이터베이스인지는 다음 절인 Wide World Importers 에서 살펴보고, 이 절에서는 모듈이 어떻게 구성되어 있는지를 먼저 본다.

모듈의 구조는 [sql-server-sample-databases-persistence] 장에서 본 AdventureWorks 의 모듈과 같다. 다른 것은 셋이다. 테스트 코드의 `wideworldimporters` 패키지에는 `geography` 컬럼의 값을 SQL Server 가 돌려주는 값과 비교하는 `Geography_PersistenceIT` 가 있다. 스키마의 패키지에는 이력 테이블을 매핑한 `CityArchive`, `PersonArchive` 같은 클래스가 함께 있다. 테스트 리소스에는 `create-test.sql` 말고도, 퍼시스터가 새 행에서 참조하는 설치된 행을 H2 에 넣는 `load-test.sql` 이 있다.

매핑한 클래스와 테스트

매핑된 슈퍼클래스는 51 개, 엔티티는 34 개다. `geography` 컬럼을 매핑하는 `Geography` 와 그 변환기는 이 모듈이 아니라 `sqlserver-samples-persistence-common` 에 있다(geography 와 바이너리). 이력 테이블 17 개를 매핑한 클래스는 엔티티가 아니다. 시스템 버전 관리 임시 테이블 절에서 그 이유를 본다. 연습문제마다 `WWI_SALES_B_01_PersistenceIT` 같은 통합 테스트가 하나씩 있다.

클래스마다 있는 통합 테스트는 모두 136 개다. 연관이 없는 엔티티와 같이 확인할 것이 없는 몇 개를 건너뛰고, 나머지는 모두 통과한다([sql-server-tests]).

Wide World Importers

Wide World Importers(WWI)는 샌프란시스코 만 지역에서 신기한 소품을 들여와 파는 가상의 도매 회사다.[1] 고객은 대부분 그 물건을 다시 개인에게 파는 가게다. 공급자에게서 물건을 사서 창고에 두고, 고객의 주문을 받으면 창고에서 꺼내 보낸다. 최근에는 초콜릿 같은 먹을거리도 팔기 시작해서, 냉장실과 냉장 트럭의 온도를 기록한다.[1]

WideWorldImporters 데이터베이스는 이 회사의 업무를 담은 OLTP 데이터베이스다. SQL Server 2016 부터 쓸 수 있는 예제로, 시스템 버전 관리 임시 테이블, JSON 함수, 시퀀스, 공간 데이터 같은 SQL Server 의 기능을 두루 쓴다. 이 장은 그 기능들이 Jakarta Persistence 의 매핑에서 어떤 모습이 되는지에 집중한다.

주문과 송장은 2013 년 1 월 1 일부터 2016 년 5 월 31 일까지 있다. 연습문제가 2015 년을 한 해의 예로 자주 쓰는 것은 이 기간 가운데 온전한 마지막 해이기 때문이다.

스키마

테이블은 업무마다 나눈 네 스키마에 있다.[2] 여기에 회사의 웹 사이트가 쓰는 뷰를 모은 `Website` 와, 모든 테이블이 쓰는 시퀀스를 모은 `Sequences` 가 더해진다.

Table 1. WideWorldImporters 의 스키마

스키마	담는 것
`Application`	사람, 도시와 주와 나라, 배송 방법과 결제 방법 같이 여러 스키마가 함께 쓰는 테이블
`Purchasing`	공급자와 그 분류, 공급자에게 낸 발주와 그 품목, 공급자와의 거래
`Sales`	고객과 그 분류와 구매 그룹, 주문과 송장과 그 품목, 고객과의 거래, 특별 할인
`Warehouse`	재고 품목과 그 그룹, 색, 포장 형태, 재고 수량, 재고의 입출고, 냉장실과 차량의 온도
`Website`	웹 사이트가 읽는 뷰 세 개. 데이터를 담지 않는다.
`Sequences`	테이블의 기본 키에 번호를 매기는 시퀀스 26 개

테이블은 48 개다. 그 가운데 17 개는 다른 테이블의 옛 행을 담는 **이력 테이블**이다(시스템 버전 관리 임시 테이블). 이력 테이블을 뺀 31 개 가운데 17 개는 시스템 버전 관리 임시 테이블, 곧 이력 테이블을 가진 테이블이다. 테이블과 뷰마다 엔티티가 하나씩 있다. 클래스는 스키마마다 패키지에 나뉘어 있고, 클래스의 이름은 테이블의 이름을 단수로 바꾼 것이다. 그래서 `Website` 의 뷰를 매핑한 `Customer` 와 `Sales` 의 테이블을 매핑한 `Customer` 는 패키지만 다르다. 두 엔티티의 이름은 Website 의 뷰 절에서 본다.

판매 쪽 테이블이 가장 크다. 주문의 품목과 송장의 품목이 각각 23 만 행 가까이 되고, 재고의 입출고도 그만큼 된다. 반면 구매 그룹이나 결제 방법 같은 참조 테이블은 몇 행뿐이다.

테이블이 이어지는 형태

업무의 테이블은 대부분 `Application` 의 테이블을 참조한다. 고객과 공급자는 배송할 도시와 우편을 받을 도시로 `Application.Cities` 를 참조하고, 도시는 `StateProvinces` 를, 주는 `Countries` 를 참조한다. 주문은 고객을, 송장은 고객과 주문을 참조하고, 주문과 송장과 발주의 품목은 모두 `Warehouse.StockItems` 를 참조한다. 세 스키마의 거래 테이블, `Sales.CustomerTransactions`, `Purchasing.SupplierTransactions`, `Warehouse.StockItemTransactions` 는 `Application.TransactionTypes` 로 거래의 종류를 나눈다.

모든 테이블의 기본 키는 컬럼 하나다.[2] 복합 키가 없으므로 `@IdClass` 도 없다. [sql-server-adventureworks-persistence] 장의 AdventureWorks 와 다른 점이다.

외래 키는 98 개이고, 그 가운데 41 개가 `Application.People` 을 참조한다. 그중 29 개는 `LastEditedBy` 컬럼이다. 이력 테이블이 아닌 31 개 테이블은, 냉장실과 차량의 온도를 담는 두 테이블만 빼고 모두 그 행을 마지막으로 고친 사람을 `LastEditedBy` 에 기재한다. `People` 자신도 그렇다. 그래서 `Person` 은 이 데이터베이스에서 가장 많이 참조되는 엔티티다. Application.People 과 외래 키 절에서 이 외래 키들을 매핑한 방법을 살펴본다.

이력 테이블을 뺀 31 개 테이블을 한 그림에 그리면 책의 쪽보다 훨씬 넓어지므로, 서로 이어진 테이블을 작은 묶음으로 나누어 그렸다. 여러 묶음에 나오는 테이블도 있다. 그림에는 같은 묶음 안의 외래 키만 그렸다. 그래서 `LastEditedBy` 가 `Application.People` 을 참조하는 외래 키는 첫 그림에만 보인다. 외래 키가 없는 `Warehouse.ColdRoomTemperatures` 와 `Warehouse.VehicleTemperatures` 는 그리지 않았다.

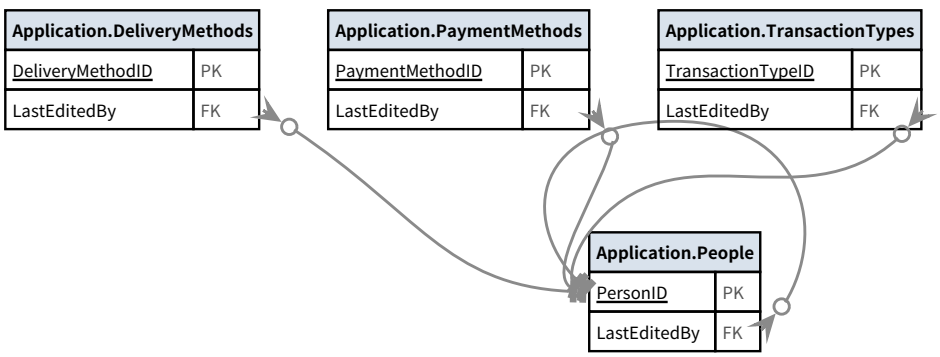


Figure 1. 사람과 공통 참조 테이블

Table 2. 사람과 공통 참조 테이블 묶음

테이블	담는 것	행
`Application.People`	직원, 고객과 공급자의 연락 담당자인 사람	1,111
`Application.DeliveryMethods`	트럭, 우편, 직접 수령, 택배 같은 배송 방법	10
`Application.PaymentMethods`	현금, 수표, 계좌 이체 같은 결제 방법	4
`Application.TransactionTypes`	청구서, 대변표 같은 고객과 공급자, 재고의 거래 종류	13

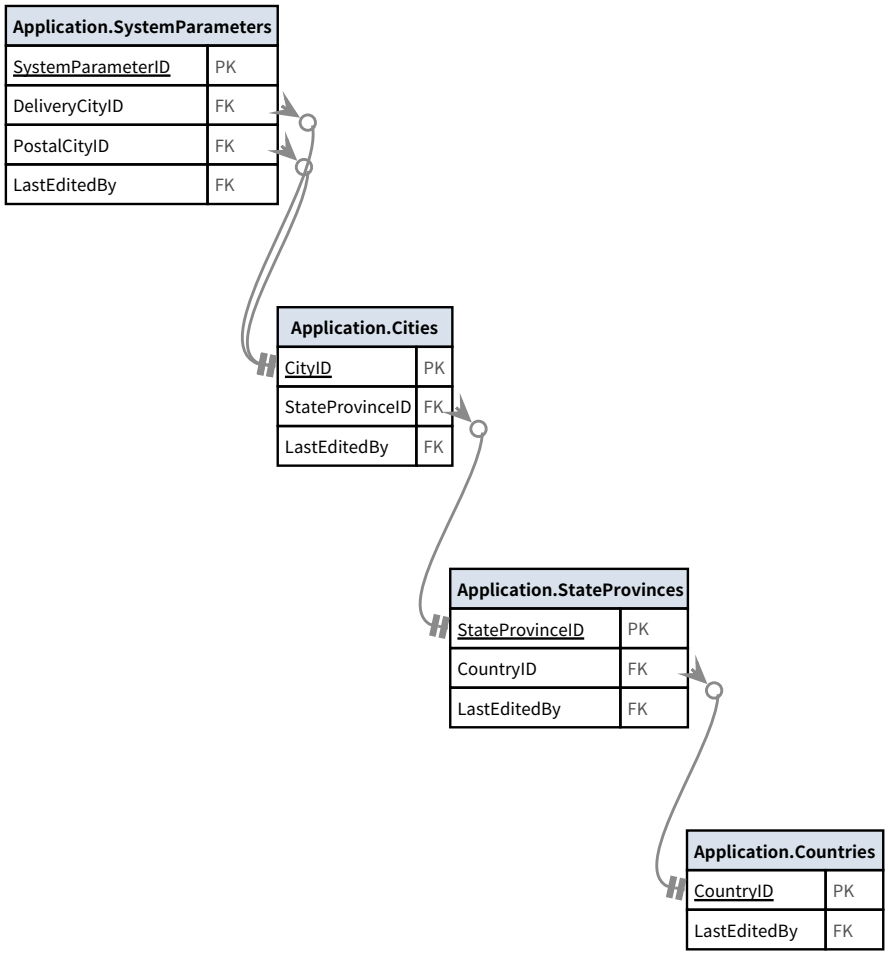


Figure 2. 도시와 주와 나라

Table 3. 도시와 주와 나라 묶음

테이블	담는 것	행
Application.Cities	주소에 쓰이는 도시와 그 위치, 인구	37,940
Application.StateProvinces	도시가 속한 주와 도, 그 경계와 인구	53
Application.Countries	주와 도가 속한 나라, 그 경계와 인구, 지역	190
Application.SystemParameters	회사의 주소 같은 시스템 전체의 설정 한 행	1

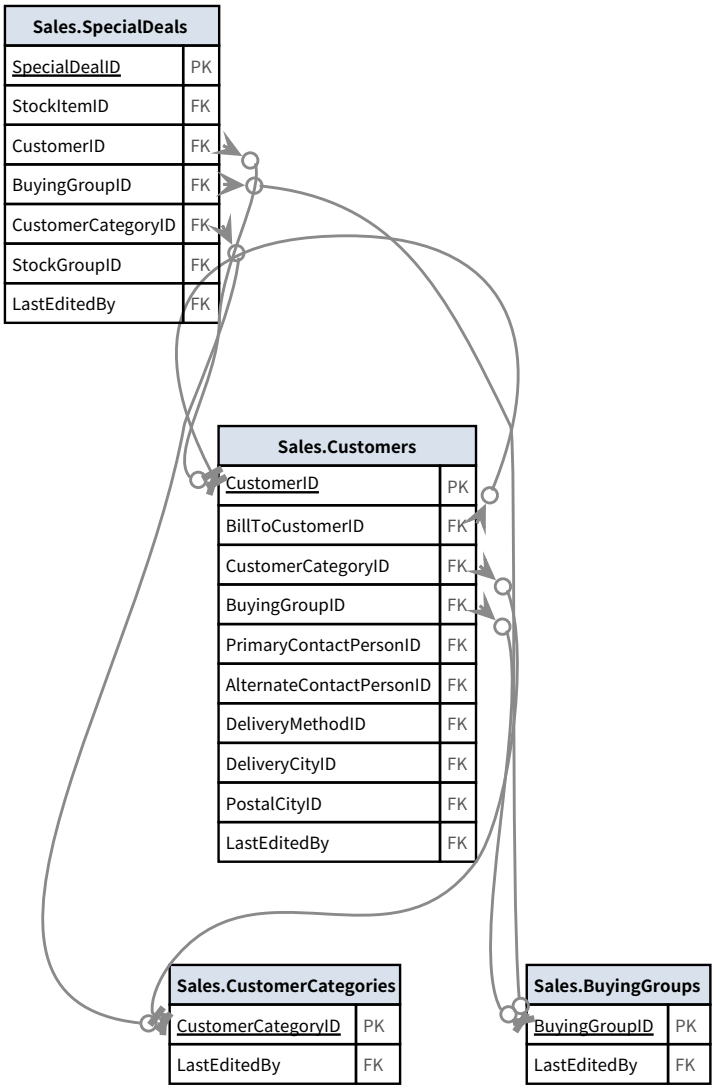


Figure 3. 고객

Table 4. 고객 묶음

테이블	담는 것	행
Sales.Customers	단체나 개인인 고객과 그 연락 담당자, 주소, 신용 한도	663
Sales.CustomerCategories	식당, 카페, 슈퍼마켓 같은 고객의 범주	8
Sales.BuyingGroups	고객이 속해 함께 구매하는 구매 그룹	2
Sales.SpecialDeals	고객이나 그 범주, 구매 그룹에 주는 특별 가격과 할인	2

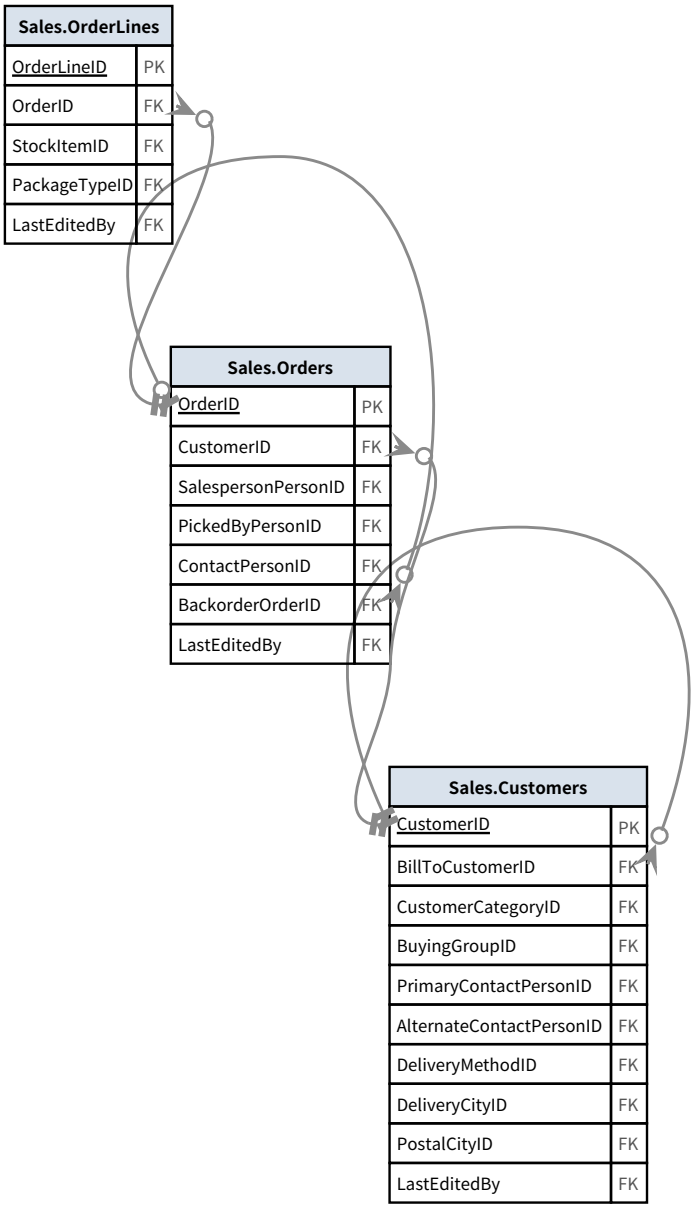


Figure 4. 주문

Table 5. 주문 묶음

테이블	담는 것	행
Sales.Orders	고객의 주문과 그 날짜, 담당 영업 사원	73,595
Sales.OrderLines	주문의 품목마다 재고 품목, 수량, 단가	231,412

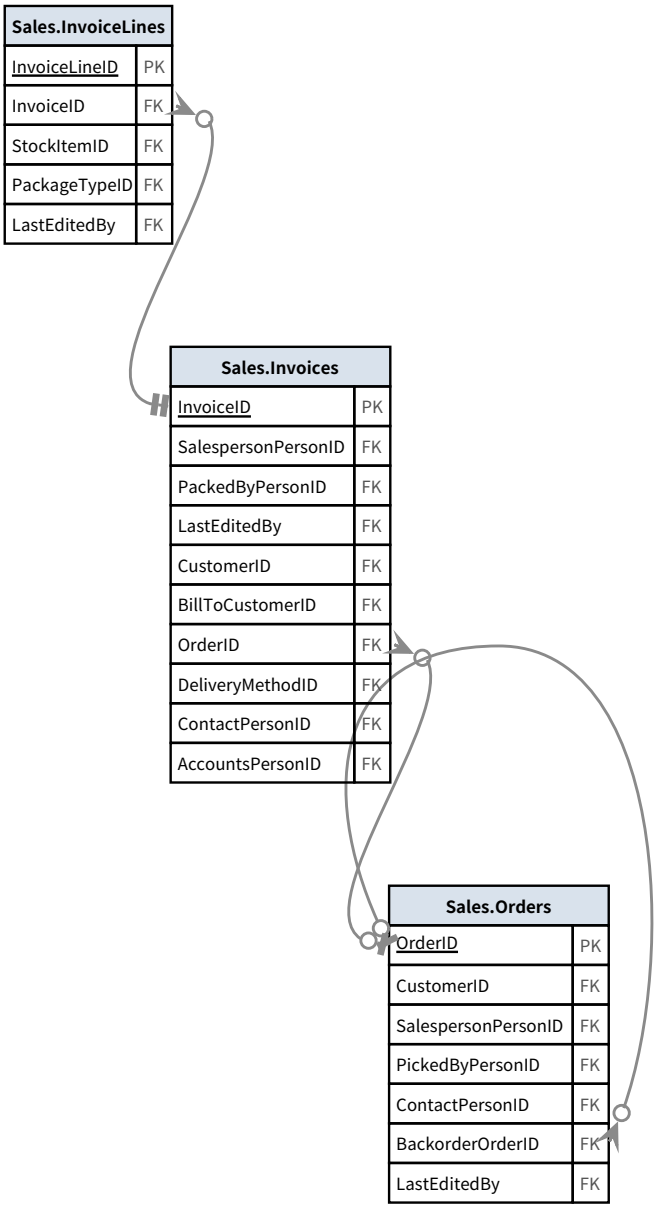


Figure 5. 송장

Table 6. 송장 묶음

테이블	담는 것	행
Sales.Invoices	고객에게 보낸 송장과 배송 정보	70,510
Sales.InvoiceLines	송장의 품목마다 재고 품목, 수량, 단가, 세금, 이익	228,265

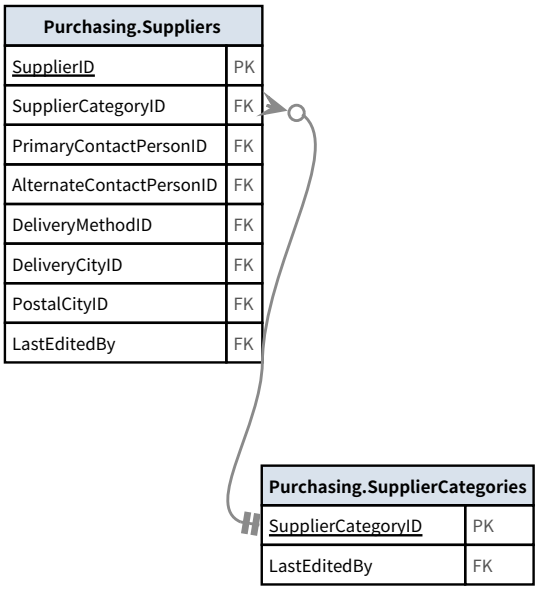


Figure 6. 공급자

Table 7. 공급자 묶음

테이블	담는 것	행
Purchasing.Suppliers	공급자와 그 연락 담당자, 주소, 결제 조건	13
Purchasing.SupplierCategories	장난감, 의류, 포장재 같은 공급자의 범주	9

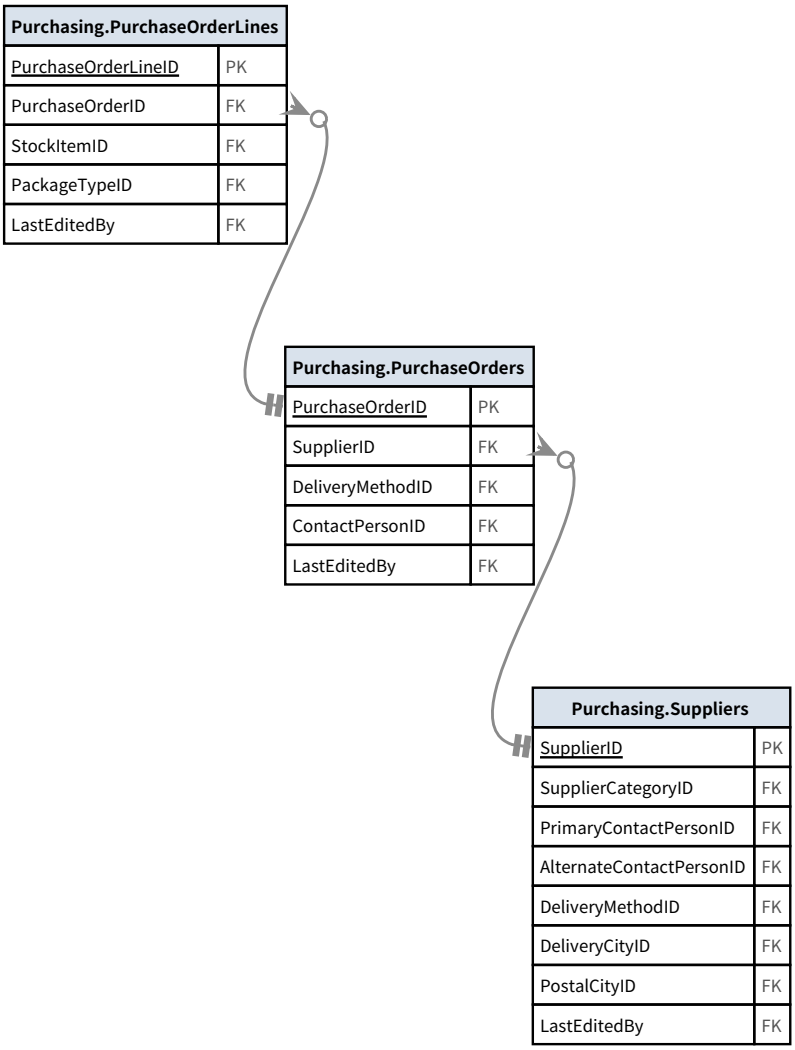


Figure 7. 발주

Table 8. 발주 묶음

테이블	담는 것	행
Purchasing.PurchaseOrders	공급자에게 낸 발주와 그 날짜, 배송 방법, 담당자	2,074
Purchasing.PurchaseOrderLines	발주의 품목마다 재고 품목, 주문 수량, 받은 수량, 단가	8,367

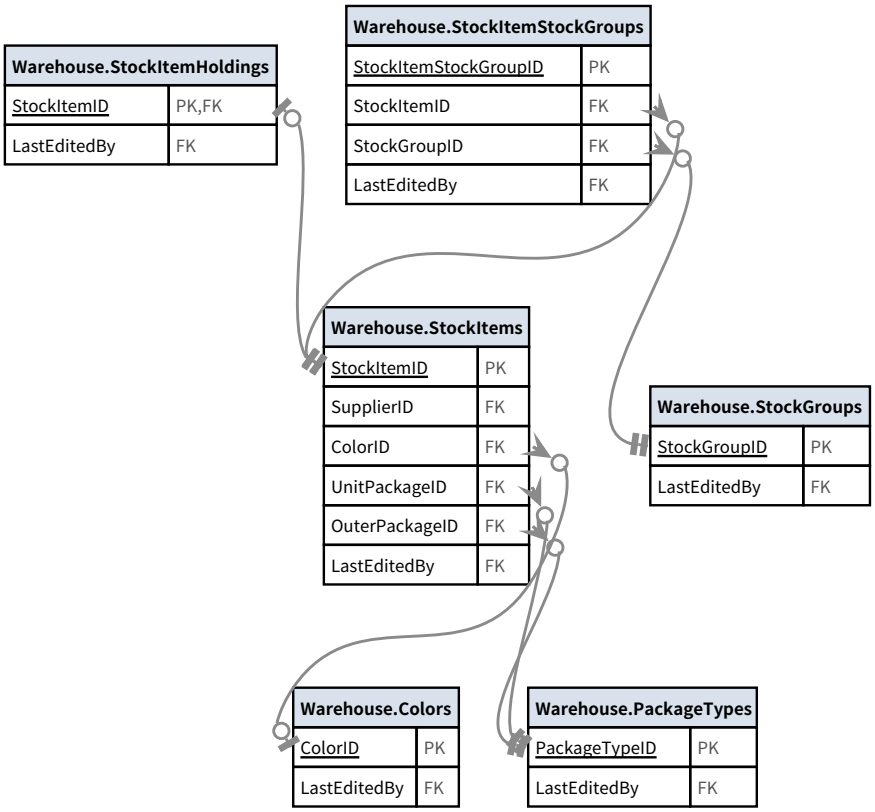


Figure 8. 재고 품목

Table 9. 재고 품목 묶음

테이블	담는 것	행
Warehouse.StockItems	파는 재고 품목과 그 공급자, 색, 포장, 가격, 세율	227
Warehouse.StockGroups	장난감, 의류, 머그잔 같은 재고 품목의 묶음	10
Warehouse.StockItemStockGroups	재고 품목과 그 묶음을 잇는 행	442
Warehouse.StockItemHoldings	재고 품목마다 지금의 재고 수량, 보관 위치처럼 이력을 남기지 않는 값	227
Warehouse.Colors	재고 품목의 색	36
Warehouse.PackageTypes	낱개, 상자, 팔레트 같은 포장 단위	14

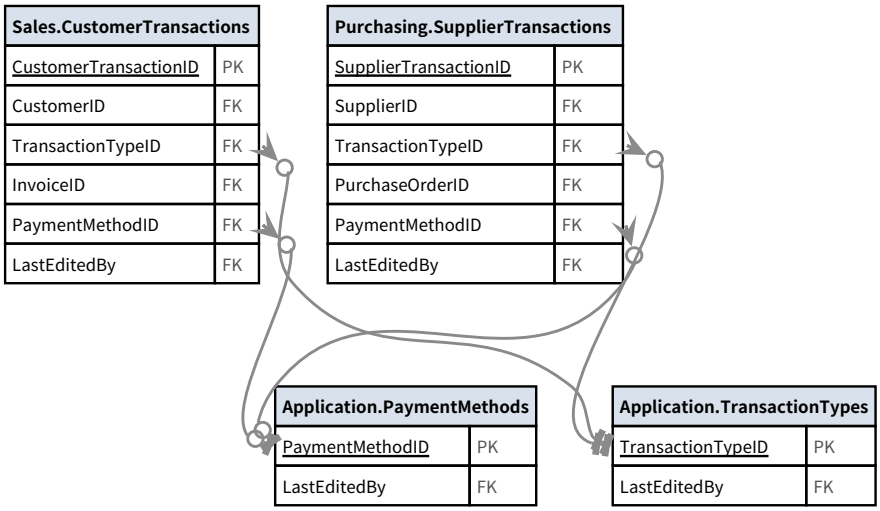


Figure 9. 고객과 공급자의 거래

Table 10. 고객과 공급자의 거래 묶음

테이블	담는 것	행
Sales.CustomerTransactions	고객과 주고받은 청구와 수금 같은 거래	97,147
Purchasing.SupplierTransactions	공급자와 주고받은 청구와 지급 같은 거래	2,438

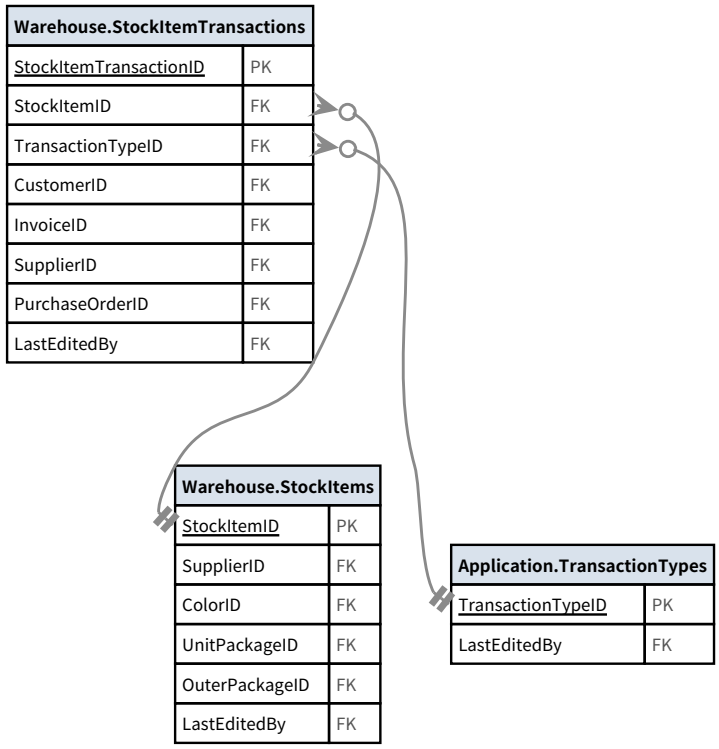


Figure 10. 재고의 입출고

Table 11. 재고의 입출고 묶음

테이블	담는 것	행
`Warehouse.StockItemTransactions`	재고 품목이 들고 난 모든 거래	236,667

그리지 않은 두 테이블은 다음과 같다.

Table 12. 그림에 없는 테이블

테이블	담는 것	행
`Warehouse.ColdRoomTemperatures`	냉장실의 센서마다 마지막으로 잰 온도	4
`Warehouse.VehicleTemperatures`	배송 차량의 냉장 센서가 잰 온도 기록	65,998

시스템 버전 관리 임시 테이블

SQL Server 의 **시스템 버전 관리 임시 테이블**(system-versioned temporal table)은 행의 지난 판을 데이터베이스가 스스로 남기는 테이블이다. 행을 고치거나 지우면, SQL Server 가 고치기 전의 행을 그 테이블에 딸린 이력 테이블(history table)에 옮긴다.[3] 행이 언제부터 언제까지 유효했는지는 두 기간 컬럼(period

column)에 기재된다. WideWorldImporters 에서는 두 컬럼의 이름이 늘 `ValidFrom` 과 `ValidTo` 이고, 이력 테이블의 이름은 원래 테이블의 이름에 `_Archive` 를 붙인 것이다.

31 개 테이블 가운데 17 개가 임시 테이블이다. 주로 자주 바뀌지 않는 참조 테이블과, 고객과 공급자와 재고 품목처럼 업무의 중심이 되는 테이블이다. 주문과 송장, 거래처럼 한 번 기록하면 거의 고치지 않는 테이블은 임시 테이블이 아니다.

Table 13. WideWorldImporters 의 임시 테이블과 이력 테이블의 행 수

임시 테이블	현재 행	이력 행	이력 클래스
Application.Cities	37,940	28	CityArchive
Application.Countries	190	37	CountryArchive
Application.DeliveryMethods	10	1	DeliveryMethodArchive
Application.PaymentMethods	4	1	PaymentMethodArchive
Application.People	1,111	961	PersonArchive
Application.StateProvinces	53	104	StateProvinceArchive
Application.TransactionTypes	13	1	TransactionTypeArchive
Purchasing.SupplierCategories	9	1	SupplierCategoryArchive
Purchasing.Suppliers	13	13	SupplierArchive
Sales.BuyingGroups	2	0	BuyingGroupArchive
Sales.CustomerCategories	8	1	CustomerCategoryArchive
Sales.Customers	663	51	CustomerArchive
Warehouse.ColdRoomTemperatures	4	3,654,736	ColdRoomTemperatureArchive
Warehouse.Colors	36	1	ColorArchive
Warehouse.PackageTypes	14	0	PackageTypeArchive
Warehouse.StockGroups	10	1	StockGroupArchive
Warehouse.StockItems	227	444	StockItemArchive

`Warehouse.ColdRoomTemperatures` 가 눈에 띈다. 냉장실의 센서 네 개가 십여 초마다 온도를 기록하는데, 테이블에는 센서마다 마지막 기록 한 행씩, 네 행만 있다. 새 기록은 행을 더하지 않고 그 센서의 행을 고치기 때문이다. 그래서 2015 년 12 월 20 일부터의 기록 365 만여 개는 모두 이력 테이블에 있다. 임시 테이블이 고친 이력을 남기는 데 그치지 않고, 시계열 데이터를 담는 데에도 쓰인 예다.

기간 컬럼

기간 컬럼은 SQL Server 가 쓴다. `INSERT` 나 `UPDATE` 가 이 컬럼에 값을 넣으려 하면 SQL Server 는 오류 13536 이나 13537 로 거절한다. **그래서 매핑된 슈퍼클래스는 두 컬럼을 읽기 전용으로 매핑한다.**

MappedBuyingGroup 의 validFrom 과 validTo

```
// 읽기 전용: GENERATED ALWAYS 인 기간 컬럼이다. SQL Server 가 값을 정하고, INSERT
// (오류 13536) 나 UPDATE (오류 13537) 가 이 컬럼에 주는 값은 거절한다
@Basic(optional = false)
@Column(name = COLUMN_NAME_VALID_FROM,
        nullable = false,
        insertable = false, ❶
        updatable = false
)
private LocalDateTime validFrom;

// 읽기 전용: GENERATED ALWAYS 인 기간 컬럼이다. SQL Server 가 값을 정하고, INSERT
// (오류 13536) 나 UPDATE (오류 13537) 가 이 컬럼에 주는 값은 거절한다
@Basic(optional = false)
@Column(name = COLUMN_NAME_VALID_TO,
        nullable = false,
        insertable = false,
        updatable = false
)
private LocalDateTime validTo;
```

❶ 구현체는 `INSERT` 와 `UPDATE` 에 이 컬럼을 넣지 않는다.

`datetime2` 는 시간대가 없는 날짜와 시각이므로 속성의 형은 `LocalDateTime` 이다([sql-server-types]). SQL Server 는 행을 삽입하거나 수정한 트랜잭션의 시작 시각을 UTC 로 ValidFrom 에 기재한다. ValidTo 는 현재 행에서는 최댓값이고, 이력 행에서는 해당 판을 종료한 트랜잭션의 시작 시각이다.[3] `LocalDateTime` 으로 읽은 값도 UTC 의 시각이다.

컬럼은 `NOT NULL` 이고 `nullable = false` 로 매핑했지만, `@NotNull` 은 두지 않았다. 새 엔티티를 저장할 때 이 속성은 아직 `null` 이다. 값은 SQL Server 가 행을 넣을 때 정하기 때문이다. `@NotNull` 이 있으면 영속성 유닛의 검증이 저장하기 전에 그 `null` 을 거절한다. JSON 과 계산 컬럼 절의 계산 컬럼도 같은 이유로 `@NotNull` 이 없다.

접근자도 다르다. `getValidFrom()` 은 `public` 이지만 `setValidFrom(LocalDateTime)` 은 `protected` 다. 값을 쓰는 것은 데이터베이스이므로, 애플리케이션이 이 속성을 바꿀 일은 없다.

읽기 전용인 속성이 늘 그렇듯, 엔티티를 고쳐 `UPDATE` 가 나간 뒤에도 영속성 컨텍스트에 있는 엔티티의 `validFrom` 은 고치기 전의 값 그대로다. SQL Server 가 새로 정한 값이 필요하면 `EntityManager.refresh(Object)` 로 행을 다시 읽는다.

이력 테이블

이력 테이블은 원래 테이블과 컬럼이 같다. 기간 컬럼도 똑같이 있지만, 이력 테이블에서는 보통의 `datetime2` 컬럼이다.

이력 테이블에는 기본 키가 없다. 인덱스는 `ValidTo` 와 `ValidFrom` 의 클러스터 인덱스 하나뿐이고, 그 인덱스도 유일하지 않다. 원래 테이블의 기본 키 컬럼, 이를테면 `BuyingGroupID` 는 이력 테이블에도 있지만, 한 행의 여러 판이 들어가므로 유일하지 않다.

기본 키 컬럼과 `ValidFrom` 을 함께 쓰면 유일할 것 같지만, 그렇지도 않다.

`Application.People_Archive` 에는 `PersonID` 와 `ValidFrom` 이 같은 행이 두 개씩인 경우가 세 번 있다. `PersonID` 가 8 인 `Anthony Grosse` 의 두 판은 모두 2013 년 11 월 25 일 08:00 에 시작한다. 그 가운데 하나는 `ValidTo` 도 같은 시각이다. 한 트랜잭션 안에서 같은 행을 두 번 고치면, 첫 번째 수정의 결과는 한순간도 유효하지 않았던 판으로 이력에 남는다.

식별자가 없으면 엔티티가 될 수 없다. 그래서 이력 테이블을 매핑한 매핑된 슈퍼클래스에는 `@Id` 가 없고, 그 클래스를 상속한 클래스는 엔티티가 아니다. README 는 이런 클래스를 **키 없는 클래스**(keyless class)라 부른다([sql-server-identifiers]).

BuyingGroupArchive

```
public class BuyingGroupArchive extends MappedBuyingGroupArchive {

    @SuppressWarnings({"java:S107"}) // 메서드의 매개변수가 너무 많아서는 안 된다
    public BuyingGroupArchive(final Integer buyingGroupId,
                              final String buyingGroupName,
                              final Integer lastEditedBy,
                              final LocalDateTime validFrom,
                              final LocalDateTime validTo) {
        super();
        setBuyingGroupId(buyingGroupId);
        setBuyingGroupName(buyingGroupName);
        setLastEditedBy(lastEditedBy);
        setValidFrom(validFrom);
        setValidTo(validTo);
    }
}
```

`@Entity` 가 없으므로 영속성 유닛은 이 클래스를 모른다. `persistence.xml` 에도 이 클래스는 없다. 그래도 매핑된 슈퍼클래스의 상수와 애너테이션은 그대로 쓸모가 있다. 컬럼의 이름과 길이를 담은 상수로 SQL 을 쓰고, 공개된 생성자에 모든 컬럼을 테이블이 선언한 순서대로 넘겨 행 하나를 담는 객체를 생성한다. `@ConstructorResult` 의 대상이 되거나, 네이티브 쿼리의 결과를 옮겨 담는 클래스다.

이력 테이블의 매핑된 슈퍼클래스는 기간 컬럼뿐 아니라 모든 컬럼을 읽기 전용으로 매핑한다. 이력 테이블에 행을 넣고 고치는 것은 원래 테이블의 시스템 버전 관리뿐이다. SQL Server 는 이력 테이블에 대한 `INSERT` 를

오류 13559 로, `UPDATE` 를 오류 13561 로 거절한다. `NOT NULL` 인 컬럼에는 `@NotNull` 도 둔다.

키 없는 클래스의 통합 테스트는 엔티티를 찾지 않고, 매핑된 슈퍼클래스가 이름을 아는 컬럼을 모두 네이티브 쿼리로 읽는다.

_Keyless_PersistenceIT 의 columns()

```
void columns() {
    final var columns = columnNames(); ❶
    assertThat(columns).isNotEmpty();
    final var sql = "SELECT TOP (1) " + String.join(", ", columns)
                    + " FROM " + constant("TABLE_SCHEMA") + "." + constant("TABLE_NAME");
    ___Persistence_TestUtils.applyInTransactionAndRollback(em -> {
        final var rows = em.createNativeQuery(sql).getResultList();
        for (final var row : rows) {
            assertThat(columns.length == 1 ? new Object[]{row} : (Object[]) row)
                    .hasSize(columns.length);
        }
        return null;
    });
}
```

❶ 매핑된 슈퍼클래스의 `COLUMN_NAME_` 으로 시작하는 상수의 값들.

`SELECT TOP (1)` 은 SQL Server 가 처음 몇 행만 돌려주게 하는 구문이다. 행을 하나만 읽어도 컬럼의 이름이 틀렸다면 SQL Server 가 쿼리를 거절하므로, 이것으로 매핑된 슈퍼클래스의 컬럼 이름을 모두 확인한다.

지난 판을 읽는 쿼리

이력을 읽는 데 이력 테이블을 직접 쿼리할 일은 드물다. SQL Server 는 임시 테이블을 쿼리할 때 `FROM` 의 테이블 뒤에 `FOR SYSTEM_TIME` 절을 두어 어느 시점이나 기간의 행을 읽게 한다.[3]

Table 14. FOR SYSTEM_TIME 의 형태

절	읽는 행
`AS OF t`	`t` 에 유효했던 행. `ValidFrom <= t AND ValidTo > t`
`FROM t1 TO t2`	`t1` 과 `t2` 사이에 한때라도 유효했던 행. `ValidFrom < t2 AND ValidTo > t1`
`BETWEEN t1 AND t2`	`FROM … TO` 에 `t2` 에 시작한 행을 더한 것. `ValidFrom <= t2 AND ValidTo > t1`
`CONTAINED IN (t1, t2)`	`t1` 과 `t2` 사이에 시작하고 끝난 행. `ValidFrom >= t1 AND ValidTo <= t2`
`ALL`	현재 행과 이력 행 모두

SQL Server 는 이 절을 받으면 현재 테이블과 이력 테이블을 함께 읽는다. 쿼리를 쓰는 쪽은 이력 테이블의 이름을 몰라도 된다.

프로젝트의 연습문제 `WWI_SALES_I_04` 는 고객 890 의 신용 한도가 바뀐 이력을 `FOR SYSTEM_TIME ALL` 로 읽는다. 결과는 세 행이다. 그런데 `Sales.Customers_Archive` 에는 이 고객의 행이 셋, 현재 행까지 넷이다.

빠진 것은 한도가 4410.00 인 판이다. 그 판은 `ValidFrom` 과 `ValidTo` 가 모두 2014 년 7 월 1 일 16:00 이다. 한 트랜잭션 안에서 4410.00 으로 고친 뒤 다시 4630.50 으로 고친 것이다. 기간 컬럼에는 트랜잭션이 시작한 시각이 들어가므로, 첫 번째 수정의 판은 시작과 끝이 같다. `FOR SYSTEM_TIME` 은 이렇게 유효했던 시간이 없는 판을 돌려주지 않는다.[4] 이력 테이블을 직접 읽어야만 보이는 판이다.

Query Language 에는 `FOR SYSTEM_TIME` 에 해당하는 것이 없다. 엔티티 `Customer` 로 쿼리하면 늘 현재 행만 읽는다. **지난 판이 필요하면 SQL 을 네이티브 쿼리로 보낸다.** 프로젝트의 `WWI_SALES_A_02` 연습문제가 그 예로, 2015 년 1 월 1 일의 고객 테이블과 지금의 고객 테이블을 조인해 신용 한도가 바뀐 고객을 찾는다. 프로젝트의 연습문제 `WWI_WAREHOUSE_A_01` 는 `FROM … TO` 로 냉장실의 하루치 기록을 읽는다.

키 없는 클래스는 이력 테이블만 매핑하지만, `FOR SYSTEM_TIME` 의 결과도 컬럼이 같으므로 그 결과를 키 없는 클래스의 생성자에 넘겨 담을 수 있다. 현재 행이든 지난 판이든 같은 형태의 객체로 다루게 된다.

시퀀스

WideWorldImporters 의 기본 키는 대부분 시퀀스로 번호를 매긴다. 테이블마다 `Sequences` 스키마에 그 테이블의 이름을 딴 시퀀스가 있고, 기본 키 컬럼의 기본값이 그 시퀀스의 다음 값이다. `Sales.BuyingGroups.BuyingGroupID` 의 기본값은 `NEXT VALUE FOR [Sequences].[BuyingGroupID]` 이다. `INSERT` 가 `BuyingGroupID` 를 빼면 SQL Server 가 시퀀스에서 번호를 받아 넣는다.

카탈로그는 `IDENTITY` 대신 시퀀스를 쓴 이유를, 여러 서버에 걸친 환경에서 다루기 쉽기 때문이라고 한다.[5] 시퀀스는 테이블과 따로 있는 객체이므로, 행을 넣기 전에 번호를 먼저 받을 수도 있다.

@jakarta.persistence.SequenceGenerator

기본값으로 정한 시퀀스는 매핑에도 그대로 옮겨진다.

MappedBuyingGroup 의 buyingGroupId

```
// 수정할 수 없다: 식별자 컬럼이고, Jakarta Persistence 는 엔티티의 식별자를 바꾸지 않는다
@Id
@jakarta.persistence.SequenceGenerator( ❶
        name = "WideWorldImporters.Sequences.BuyingGroupID",
        schema = "Sequences",
        sequenceName = "BuyingGroupID",
        allocationSize = 1)             ❷
@GeneratedValue(
        strategy = GenerationType.SEQUENCE,
        generator = "WideWorldImporters.Sequences.BuyingGroupID")
@Column(name = COLUMN_NAME_BUYING_GROUP_ID,
        nullable = false,
        insertable = true,              ❸
        updatable = false
)
private Integer buyingGroupId;
```

❶ 생성기의 이름은 데이터베이스와 스키마, 시퀀스의 이름을 이어 붙인 것이다.

❷ 시퀀스가 1 씩 늘어나므로 한 번에 번호 하나씩 받는다.

❸ 구현체가 시퀀스에서 받은 번호를 INSERT 에 넣는다.

`@jakarta.persistence.SequenceGenerator` 의 요소는 아래와 같다.

Table 15. @jakarta.persistence.SequenceGenerator 의 요소

요소	형	기본값	뜻
name	String	""	생성기 이름. @GeneratedValue 의 generator 가 이 이름으로 참조한다. 엔티티 클래스나 기본 키 속성에 붙였으면 비워 둘 때 엔티티 이름이다.
sequenceName	String	""	값을 얻을 데이터베이스 시퀀스의 이름. 비워 두면 구현체가 정한다.
catalog	String	""	시퀀스의 카탈로그.
schema	String	""	시퀀스의 스키마.
initialValue	int	1	시퀀스가 값을 생성하기 시작하는 값.
allocationSize	int	50	시퀀스 번호를 할당할 때 늘리는 양.
options (3.2)	String	""	시퀀스를 만드는 DDL 문 뒤에 붙일 SQL 조각. 테이블을 만들 때만 쓴다.

`GenerationType.SEQUENCE` 로 정한 식별자는 구현체가 정한다. 구현체는 엔티티를 저장할 때 `@SequenceGenerator` 가 기재한 시퀀스에서 다음 값을 받아 식별자에 넣고, 그 값을 넣은 INSERT 를 보낸다.
컬럼의 기본값은 쓰이지 않는다. 번호를 내주는 것은 같은 시퀀스이므로, 애플리케이션이 넣은 행과 다른 곳에서 넣은 행의 번호가 겹치지 않는다.

`allocationSize` 의 기본값은 50 이다. 구현체는 시퀀스에서 값 하나를 받으면 그 값부터 `allocationSize` 개의 번호를 미리 받은 것으로 여기고, 다음 번호들을 메모리에서 매긴다. **시퀀스가 실제로 50 씩 늘어나야 이 계산이 맞는다.** `Sequences` 의 시퀀스는 모두 1 씩 늘어나므로, `allocationSize` 가 50 이면 구현체가 메모리에서 매긴 번호가 다른 곳에서 시퀀스로 받은 번호와 겹친다. 그래서 `allocationSize = 1` 로 매번 시퀀스에서 번호를 받는다. 엔티티를 하나 저장할 때마다 시퀀스를 한 번 더 읽지만, 번호가 겹치지는 않는다.

세 테이블이 함께 쓰는 TransactionID

시퀀스 하나를 여러 테이블이 함께 쓰기도 한다. 고객과의 거래, 공급자와의 거래, 재고의 입출고를 담는 세 테이블은 모두 `Sequences.TransactionID` 에서 번호를 받는다.[5] 그래서 세 테이블의 번호는 서로 겹치지 않는다. `Sales.CustomerTransactions` 의 처음 세 행은 2, 5, 7 번이고, 그 사이의 번호는 다른 두 테이블의 행이 가져갔다.

MappedCustomerTransaction 의 customerTransactionId

```
// 수정할 수 없다: 식별자 컬럼이고, Jakarta Persistence 는 엔티티의 식별자를 바꾸지 않는다
@Id
@SequenceGenerator(name = "WideWorldImporters.Sequences.TransactionID", schema = "Sequences",
                   sequenceName = "TransactionID",
                   allocationSize = 1)
@GeneratedValue(strategy = GenerationType.SEQUENCE,
                generator = "WideWorldImporters.Sequences.TransactionID")
@Column(name = COLUMN_NAME_CUSTOMER_TRANSACTION_ID,
        nullable = false,
        insertable = true,
        updatable = false
)
private Integer customerTransactionId;
```

`MappedStockItemTransaction` 과 `MappedSupplierTransaction` 도 똑같은 `@SequenceGenerator` 를 기재한다. 생성기의 이름은 영속성 유닛 안에서 하나의 생성기를 뜻한다. **세 클래스가 같은 이름으로 같은 시퀀스를 기재하므로, 세 엔티티는 같은 생성기에서 번호를 받는다.** 정의가 같으면 구현체는 이것을 받아들인다. 모듈의 통합 테스트가 Hibernate ORM 으로 영속성 유닛을 읽어 이를 확인한다.

IDENTITY 를 쓰는 두 테이블

시퀀스를 쓰지 않는 테이블이 둘 있다. 냉장실과 차량의 온도를 담는 `Warehouse.ColdRoomTemperatures` 와 `Warehouse.VehicleTemperatures` 다. 두 테이블의 기본 키는 `bigint` 인 `IDENTITY` 컬럼이다. 카탈로그는 이 두 테이블을 메모리 최적화 테이블로 바꿀 수 있게 설계했고, SQL Server 2016 의 메모리 최적화 테이블은 시퀀스를 쓰지 못했기 때문이라고 한다.[6]

MappedColdRoomTemperature 의 coldRoomTemperatureId

```
// 수정할 수 없다: SQL Server 가 수정하지 않는 IDENTITY 컬럼이고(오류 8102), 식별자이기도 하다.
// @GeneratedValue(strategy = IDENTITY) 이므로 구현체는 INSERT 에서 이 컬럼을 뺀다
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = COLUMN_NAME_COLD_ROOM_TEMPERATURE_ID,
        nullable = false,
        insertable = true,
        updatable = false
)
private Long coldRoomTemperatureId;
```

`GenerationType.IDENTITY` 로 정한 식별자는 데이터베이스가 행을 넣을 때 정한다. 구현체는 식별자를 뺀 `INSERT` 를 보내고, SQL Server 가 매긴 번호를 돌려받아 엔티티에 넣는다. `bigint` 이므로 속성의 형은 `Long` 이다.

Application.People 과 외래 키

`Application.People` 은 직원과 고객의 연락처, 공급자의 연락처를 모두 담는다. 로그인할 수 있는 사람의 정보도 이 테이블에 있다. 그래서 다른 테이블이 사람을 참조할 일이 생기면 언제나 `People` 을 참조한다. 외래 키 98 개 가운데 41 개가 `People` 을 참조하고, 그 가운데 29 개가 `LastEditedBy` 다.

엔티티에서 외래 키는 모두 `@ManyToOne` 이다. 98 개 모두 `fetch = FetchType.LAZY` 이고, 반대 방향의 `@OneToMany` 는 하나도 없다.

LastEditedBy

LastEditedBy 도 다른 외래 키처럼 매핑된 슈퍼클래스의 Integer 속성이 읽기만 하고, 엔티티의 @ManyToOne 이 컬럼을 쓴다([sql-server-mapping-rules]).

MappedBuyingGroup 의 lastEditedBy

```
// 읽기 전용: 이 컬럼을 쓰는 매핑은 상속한 엔티티가 가진다. 클래스의 문서를 보라
@NotNull
@Basic(optional = false)
@Column(name = COLUMN_NAME_LAST_EDITED_BY,
        nullable = false,
        insertable = false,
        updatable = false
)
private Integer lastEditedBy;
```

BuyingGroup 의 lastEditedByPerson

```
public void setLastEditedByPerson(final Person lastEditedByPerson) {
    this.lastEditedByPerson = lastEditedByPerson;
    setLastEditedBy(lastEditedByPerson == null ? null : lastEditedByPerson.getPersonId()); ❶
}

@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_LAST_EDITED_BY,
            referencedColumnName = MappedPerson.COLUMN_NAME_PERSON_ID,
            nullable = false,
            insertable = true,
            updatable = true
)
private Person lastEditedByPerson;
```

❶ 연관을 바꾸면 읽기 전용인 기본 속성도 함께 바꾼다.

이 매핑에서는 식별자만 필요할 때 Person 을 읽지 않아도 된다. getLastEditedBy() 는 컬럼의 값을 그대로 돌려준다. LAZY 인 연관은 getLastEditedByPerson() 으로 그 사람의 정보가 필요해진 때에야 읽는다.

연관이 LAZY 인 것은 이 데이터베이스에서 특히 중요하다. @ManyToOne 의 기본값은 EAGER 다. Invoice 에는 @ManyToOne 이 아홉 개 있고, 그 가운데 다섯 개가 Person 이다. 모두 EAGER 였다면 송장 하나를 읽을 때마다 사람 다섯 명과 고객 둘, 주문과 배송 방법을 함께 읽고, 그 사람과 고객이 다시 자신의 LastEditedBy 를 따라 읽는다.

Person 도 LastEditedBy 로 Person 을 참조한다. 자기 자신을 참조하는 연관이지만 매핑은 다른 엔티티와 같다. Customer 의 billToCustomer 와 Order 의 backorderOrder 도 자기 자신을 참조한다.

연관의 이름

연관의 이름은 외래 키 컬럼의 이름에서 나온다. 컬럼의 이름이 `ID` 로 끝나면 `ID` 를 뗀 것이 연관의 이름이다. `ID` 로 끝나지 않는 `LastEditedBy` 는 컬럼의 이름이 이미 기본 속성 `lastEditedBy` 의 이름이므로, 참조하는 엔티티의 이름을 붙여 `lastEditedByPerson` 이 된다.

Table 16. 외래 키 컬럼과 연관의 이름

컬럼	기본 속성	연관
`LastEditedBy`	`lastEditedBy`	`lastEditedByPerson`
`SalespersonPersonID`	`salespersonPersonId`	`salespersonPerson`
`ContactPersonID`	`contactPersonId`	`contactPerson`
`BillToCustomerID`	`billToCustomerId`	`billToCustomer`
`BackorderOrderID`	`backorderOrderId`	`backorderOrder`
`UnitPackageID`, `OuterPackageID`	`unitPackageId`, `outerPackageId`	`unitPackage`, `outerPackage`
`DeliveryCityID`, `PostalCityID`	`deliveryCityId`, `postalCityId`	`deliveryCity`, `postalCity`

이름에 역할이 들어 있으므로, 같은 엔티티를 여러 번 참조해도 이름이 겹치지 않는다. `Invoice` 는 `Person` 을 다섯 번 참조한다.

Invoice 의 salespersonPerson, packedByPerson, lastEditedByPerson

```
@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_SALESPERSON_PERSON_ID,
            referencedColumnName = MappedPerson.COLUMN_NAME_PERSON_ID,
            nullable = false,
            insertable = true,
            updatable = true
)
private Person salespersonPerson;

@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_PACKED_BY_PERSON_ID,
            referencedColumnName = MappedPerson.COLUMN_NAME_PERSON_ID,
            nullable = false,
            insertable = true,
            updatable = true
)
private Person packedByPerson;

@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_LAST_EDITED_BY,
            referencedColumnName = MappedPerson.COLUMN_NAME_PERSON_ID,
            nullable = false,
            insertable = true,
            updatable = true
)
private Person lastEditedByPerson;
```

`Person` 을 참조하는 41 개의 연관을 이름으로 나누면 아래와 같다.

Table 17. Person 을 참조하는 연관

연관	개수	엔티티
lastEditedByPerson	29	이력 테이블과 온도를 담는 두 테이블을 뺀 모든 테이블의 엔티티
contactPerson	3	Invoice, Order, PurchaseOrder
salespersonPerson	2	Invoice, Order
primaryContactPerson	2	Customer, Supplier
alternateContactPerson	2	Customer, Supplier
pickedByPerson	1	Order
packedByPerson	1	Invoice
accountsPerson	1	Invoice

반대 방향, 곧 Person 에서 그 사람이 고친 행이나 맡은 주문으로 가는 @OneToMany 는 없다. 있다면 Person 에 컬렉션이 41 개 생긴다. 그 사람이 마지막으로 고친 송장이 필요하면 Invoice 에서 쿼리한다.

기본 키이기도 한 외래 키

외래 키가 기본 키이기도 하면 거꾸로다. Warehouse.StockItemHoldings 는 재고 품목마다 재고 수량을 담는 테이블로, 기본 키 StockItemID 가 StockItems 를 참조하는 외래 키다. 카탈로그는 이 테이블을, 자주 바뀌는 컬럼을 임시 테이블인 StockItems 에서 떼어 낸 것이라고 한다.[7] 재고 수량이 바뀔 때마다 StockItems 의 이력이 쌓이지 않게 한 것이다.

식별자는 @Id 인 기본 속성이 써야 한다. 그래서 StockItemID 는 매핑된 슈퍼클래스의 stockItemId 가 쓰고, 엔티티의 연관이 읽기 전용이다.

MappedStockItemHolding 의 stockItemId

```
// 수정할 수 없다: 식별자 컬럼이고, Jakarta Persistence 는 엔티티의 식별자를 바꾸지 않는다
@NotNull
@Id
@Column(name = COLUMN_NAME_STOCK_ITEM_ID,
        nullable = false,
        insertable = true, ❶
        updatable = false
)
private Integer stockItemId;
```

❶ 식별자인 기본 속성이 컬럼을 쓴다.

StockItemHolding 의 stockItem

```
public void setStockItem(final StockItem stockItem) {
    this.stockItem = stockItem;
    setStockItemId(stockItem == null ? null : stockItem.getStockItemId()); ❶
}

@ManyToOne(optional = false, fetch = FetchType.LAZY)
@JoinColumn(name = COLUMN_NAME_STOCK_ITEM_ID,
            referencedColumnName = MappedStockItem.COLUMN_NAME_STOCK_ITEM_ID,
            nullable = false,
            insertable = false,                                    ❷
            updatable = false
)
private StockItem stockItem;
```

❶ 연관으로 재고 품목을 정하면 식별자도 따라 바뀐다.

❷ 연관은 컬럼을 읽기만 한다.

`@MapsId` 를 쓰면 연관이 식별자를 정하게 할 수도 있다. 하지만 그러면 식별자를 정하는 일이 엔티티의 연관으로 옮겨 가고, 매핑된 슈퍼클래스만으로는 식별자를 쓸 수 없게 된다. 매핑된 슈퍼클래스에 연관을 두지 않는 이 프로젝트에서는([sql-server-mapping-rules]), 식별자인 기본 속성이 컬럼을 쓰고 연관이 읽기만 하는 쪽이 규칙에 맞는다.

Website 의 뷰

`Website` 스키마는 회사의 웹 사이트를 위한 스키마다. 데이터를 담지 않고, 웹 사이트가 쓰는 뷰와 저장 프로시저만 담는다. 웹 사이트는 업무의 테이블을 직접 읽지 않고 이 스키마를 거쳐서만 데이터에 접근한다.[8] WideWorldImporters 의 뷰는 이 스키마의 세 개뿐이다.

Website.Customers

```
CREATE VIEW Website.Customers
AS
SELECT s.CustomerID, s.CustomerName, sc.CustomerCategoryName, pp.FullName AS PrimaryContact,
       ap.FullName AS AlternateContact, s.PhoneNumber, s.FaxNumber, bg.BuyingGroupName,
       s.WebsiteURL, dm.DeliveryMethodName AS DeliveryMethod, c.CityName AS CityName,
       s.DeliveryLocation AS DeliveryLocation, s.DeliveryRun, s.RunPosition
FROM Sales.Customers AS s
LEFT OUTER JOIN Sales.CustomerCategories AS sc ON s.CustomerCategoryID = sc.CustomerCategoryID
LEFT OUTER JOIN [Application].People AS pp ON s.PrimaryContactPersonID = pp.PersonID
LEFT OUTER JOIN [Application].People AS ap ON s.AlternateContactPersonID = ap.PersonID
LEFT OUTER JOIN Sales.BuyingGroups AS bg ON s.BuyingGroupID = bg.BuyingGroupID
LEFT OUTER JOIN [Application].DeliveryMethods AS dm ON s.DeliveryMethodID = dm.DeliveryMethodID
LEFT OUTER JOIN [Application].Cities AS c ON s.DeliveryCityID = c.CityID
```

`Website.Customers` 는 고객의 행에, 식별자로 참조하던 분류와 연락처, 구매 그룹, 배송 방법, 도시를 이름으로 바꾸어 붙인 것이다. 웹 사이트는 조인 없이 고객의 정보를 한 행으로 읽는다. `Website.Suppliers` 는 같은 일을 공급자에 대해 한다.

`Website.VehicleTemperatures` 는 차량의 온도 기록을 그대로 옮기되, `FullSensorData` 만 다르다. 기록이 압축되어 있으면 `CompressedSensorData` 를 `DECOMPRESS` 로 풀어 문자열로 돌려준다. geography 와 바이너리 절에서 본 압축된 바이너리를 읽는 쪽이 풀지 않아도 되게 한 것이다.

뷰의 식별자

뷰에는 기본 키가 없다. 그래도 엔티티에는 식별자가 있어야 한다. [sql-server-identifiers] 절에서 본 대로, 이 프로젝트는 뷰가 돌려주는 모든 행에서 유일하고 `NULL` 이 아닌 식별자 컬럼을 식별자로 삼는다.

`Website.Customers` 의 식별자는 `CustomerID` 다. 뷰의 663 행에서 `CustomerID` 는 모두 다르다. `Sales.Customers` 의 기본 키를 그대로 가져왔고, 뷰의 조인은 모두 고객 쪽에서 하나의 행을 참조하는 `LEFT OUTER JOIN` 이므로 고객 하나가 두 행이 되지 않는다.

website.MappedCustomer 의 customerId

```
// 읽기 전용: SQL Server 는 이 뷰에 쓰지 않는다. 클래스의 문서를 보라
@NotNull
@Id
@Column(name = COLUMN_NAME_CUSTOMER_ID,
        nullable = false,
        insertable = false,
        updatable = false
)
private Integer customerId;
```

`@GeneratedValue` 는 없다. 뷰의 컬럼에는 기본값이 없으므로 시퀀스도 기재되지 않는다.

그런데 `Website.VehicleTemperatures` 의 식별자에는 있다.

website.MappedVehicleTemperature 의 vehicleTemperatureId

```
// 읽기 전용: SQL Server 는 이 뷰에 쓰지 않는다. 클래스의 문서를 보라
@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Column(name = COLUMN_NAME_VEHICLE_TEMPERATURE_ID,
        nullable = false,
        insertable = false,
        updatable = false
)
private Long vehicleTemperatureId;
```

뷰가 `Warehouse.VehicleTemperatures.VehicleTemperatureID` 를 그대로 선택하므로, SQL Server 의 시스템 카탈로그(`sys.columns`)는 뷰의 이 컬럼도 `IDENTITY` 라고 알려 준다. 매핑된 슈퍼클래스는 시스템 카탈로그에 기록된 것을 옮긴 것이다. 뷰에 행을 넣을 일이 없으므로 이 `@GeneratedValue` 가 쓰일 일도 없다.

읽기 전용인 뷰

매핑된 슈퍼클래스는 뷰의 모든 컬럼을 읽기 전용으로 매핑한다. `@Column` 의 `insertable` 과 `updatable` 은 식별자를 포함해 모두 `false` 다. 구현체는 엔티티의 행 전체를 `INSERT` 하거나 `UPDATE` 하는데, SQL Server 는 이 뷰들의 행 전체를 받지 않는다. `Website.Customers` 와 `Website.Suppliers` 의 행은 여러 기본 테이블에 걸쳐 있어서 오류 4405 가 난다. `Website.VehicleTemperatures` 에는 `FullSensorData` 처럼 계산한 컬럼이 있어서 오류 4406 이 나고, 기본 테이블의 `IDENTITY` 컬럼은 어떤 `UPDATE` 도 바꾸지 못한다(오류 8102). 매핑된 슈퍼클래스는 이유를 클래스의 문서에 기재해 둔다.

엔티티의 이름

뷰를 매핑한 클래스의 이름은 `Customer`, `Supplier`, `VehicleTemperature` 다. 뷰의 `Customer` 는 `Sales.Customers` 를 매핑한 `Customer` 와 이름이 같고 패키지만 다르다. [sql-server-mapping-rules] 에서 본 대로 엔티티의 이름에는 스키마가 붙으므로, 뷰의 것은 `WebsiteCustomer`, 테이블의 것은 `SalesCustomer` 이고, Query Language 에서는 이 이름으로 둘을 구별한다.

website.Customer

```
/**
 * {@code Website.Customers} 뷰를 매핑한 엔티티.
 * <p>
 * 뷰는 읽기만 하고 쓰지 않는다. 이 엔티티는 연관을 매핑하지 않는다.
 */
@jakarta.persistence.Entity(name = "WebsiteCustomer") ❶
@Table(schema = MappedCustomer.TABLE_SCHEMA, name = MappedCustomer.TABLE_NAME)
public class Customer extends MappedCustomer {

    protected Customer() {
        super();
    }
}
```

❶ 엔티티의 이름은 스키마의 이름으로 시작한다.

뷰의 엔티티에는 연관이 없다. 뷰는 이미 참조하던 행의 이름을 붙여 두었고, 뷰의 `CustomerID` 를 `Sales.Customers` 로 잇는 외래 키도 없다. 고객의 다른 정보가 필요하면 `customerId` 로 `sales` 의 `Customer` 를 찾는다. 프로젝트의 `WWI_SALES_B_03` 연습문제는 `WebsiteCustomer` 로 구매 그룹이 없는 고객을 찾는다.

Attribute Mappings

WideWorldImporters 의 테이블 48 개와 뷰 3 개에서 매핑된 슈퍼클래스가 컬럼을 매핑한 속성은 모두 589 개이고, 컬럼마다 속성이 꼭 하나씩 있다. SQL Server 의 형을 어떤 자바의 형으로 매핑하는지는 [sql-server-types] 에서 보았다. 이 절은 먼저 WideWorldImporters 가 그 표에 더하는 것을 요약하고, 그다음 WideWorldImporters 가 매핑에 더하는 형과 컬럼을 주제마다 살펴본다. 테이블을 하나씩 살펴보지는 않는다.

요약

- `int` 인 기본 키는 시퀀스에서 값을 받는다(시퀀스).
- 시스템 버전 관리 임시 테이블의 기간 컬럼 `ValidFrom` 과 `ValidTo` 는 `datetime2` 이고, 읽기 전용인 `LocalDateTime` 이다(시스템 버전 관리 임시 테이블).
- `nvarchar(max)` 에 담은 JSON 문서는 객체로 바인딩하지 않고 `String` 으로 둔다(JSON 과 계산 컬럼).
- `geography` 를 쓰는 컬럼과 바이너리 컬럼은 geography 와 바이너리 에서 본다.

---

geography 와 바이너리

geography

`geography` 는 지구 위의 점(點, point)과 선과 영역을 담는 SQL Server 의 공간 데이터 형이다. WideWorldImporters 에서는 도시의 위치(`Cities.Location`), 주와 나라의 경계선(`StateProvinces.Border`, `Countries.Border`), 고객과 공급자의 배송 위치(`DeliveryLocation`)가 `geography` 다. 도시 37,940 개는 모두 위치가 있다.

Jakarta Persistence 에는 공간 데이터를 위한 형이 없다. 구현체마다 공간 데이터를 다루는 확장이 있지만, 이 프로젝트의 매핑은 Jakarta Persistence 의 API 만 쓰므로([sql-server-mapping-rules]) 그런 확장을 쓰지 않는다. 대신 세 모듈이 함께 쓰는 `sqlserver-samples-persistence-common` 의 `Geography` 로 매핑한다([sql-server-types]). JDBC 드라이버는 `geography` 컬럼을 바이트로 주고받고, 변환기 `GeographyAttributeConverter` 가 그 바이트와 `Geography` 를 서로 바꾼다.

MappedCity 의 location

```
@Convert(converter = GeographyAttributeConverter.class) ❶
@Basic(optional = true)
@Column(name = COLUMN_NAME_LOCATION,
        nullable = true,
        insertable = true,
        updatable = true
)
private Geography location;
```

❶ 컬럼의 바이트를 `Geography` 로 바꾸는 변환기를 기재한다.

GeographyAttributeConverter

```
@Converter
public class GeographyAttributeConverter
        implements AttributeConverter<Geography, byte[]> {

    @Override
    public byte[] convertToDatabaseColumn(final Geography attribute) {
        return Optional.ofNullable(attribute).map(Geography::getBytes).orElse(null);
    }

    @Override
    public Geography convertToEntityAttribute(final byte[] dbData) {
        return Optional.ofNullable(dbData).map(Geography::decode).orElse(null);
    }
}
```

`@Converter` 에 `autoApply` 를 기재하지 않았으므로, 변환기는 `@Convert` 를 기재한 속성에만 쓰인다. `AttributeConverter` 의 두 메서드는 `null` 을 받으면 `null` 을 돌려준다.

바이트는 SQL Server 가 `geography` 를 저장하는 직렬화 형식 그대로다. Microsoft 의 문서 [MS-SSCLRT] 의 2.1 절이 이 형식을 정한다.[9] 샌프란시스코(`CityID` 30378)의 `Location` 은 22 바이트이고, 16 진수로 쓰면 `E6100000010C529ACDE330E34240DFFB1BB4D79A5EC0` 이다. 처음 네 바이트 `E6100000` 은 리틀 엔디언으로 읽은 4326, 곧 이 값이 쓰는 공간 참조 시스템(SRID)인 WGS 84 의 번호다. 그 뒤의 두 바이트는 형식의 판과 속성이고, 나머지 16 바이트는 위도 37.7749… 와 경도 -122.4194… 를 담은 `double` 두 개다.

Geography 는 이 형식을 읽고 쓰는 클래스다.

Table 18. Geography 의 메서드

메서드	하는 일
`static Geography decode(byte[])`	직렬화된 바이트를 읽는다. SRID 가 -1 인, 값이 없는 geography 면 `null` 을 돌려준다.
`static Geography point(int, double, double)`	`geography::Point` 처럼 SRID 와 위도, 경도로 점을 생성한다.
`byte[] getBytes()`	직렬화된 바이트를 돌려준다. `decode` 가 읽은 바이트와 같다.
`int getSrid()`	공간 참조 시스템의 번호를 돌려준다.
`int getNumPoints()`	`STNumPoints` 처럼 점의 수를 돌려준다.
`double getLatitude(int), double getLongitude(int)`	그 순번의 점의 위도와 경도를 돌려준다.
`String toString()`	`STAsText` 처럼 WKT(Well-Known Text)로 바꾼다. 경도가 먼저다.
`String asTextZM()`	`AsTextZM` 처럼 Z 값과 M 값까지 담은 WKT 로 바꾼다.

샌프란시스코의 `location` 을 `toString()` 으로 바꾸면 `POINT (-122.4194155 37.7749295)` 이고, SQL Server 의 `Location.STAsText()` 와 같다. 다만 SQL Server 는 좌표를 유효 숫자 15 자리나 17 자리로 쓰고, `toString()` 은 같은 `double` 로 읽히는 가장 짧은 꼴로 쓴다. 그래서 좌표의 마지막 자리가 다를 때가 있다. `equals` 와 `hashCode` 는 바이트 배열을 쓴다.

SQL Server 의 JDBC 드라이버에도 `com.microsoft.sqlserver.jdbc.Geography` 가 있지만 매핑은 그것을 쓰지 않는다. 모듈은 드라이버에 `provided` 범위로만 의존하고, `Geography` 는 드라이버 없이 [MS-SSCLRT] 의 형식을 직접 읽고 쓴다.

변환기는 영속성 유닛이 관리하는 클래스의 속성에만 쓰인다. 시스템 버전 관리 임시 테이블 절의 키 없는 클래스는 엔티티가 아니므로 변환기가 적용되지 않는다. 그래서 키 없는 클래스는 생성자에서 geography 컬럼의 바이트를 받아 직접 Geography 로 읽는다.

CityArchive 의 생성자

```
@SuppressWarnings({"java:S107"}) // 메서드의 매개변수가 너무 많아서는 안 된다
public CityArchive(final Integer cityId,
                   final String cityName,
                   final Integer stateProvinceId,
                   final byte[] location,                                    ❶
                   final Long latestRecordedPopulation,
                   final Integer lastEditedBy,
                   final LocalDateTime validFrom,
                   final LocalDateTime validTo) {
    super();
    setCityId(cityId);
    setCityName(cityName);
    setStateProvinceId(stateProvinceId);
    setLocation(Optional.ofNullable(location).map(Geography::decode).orElse(null)); ❷
    setLatestRecordedPopulation(latestRecordedPopulation);
    setLastEditedBy(lastEditedBy);
    setValidFrom(validFrom);
    setValidTo(validTo);
}
```

❶ 네이티브 쿼리가 읽은 컬럼의 바이트를 받는다.

❷ 변환기와 같이 Geography.decode(byte[]) 로 읽는다.

모듈의 Geography_PersistenceIT 는 설치된 값으로 이 매핑을 확인한다. 엔티티 여섯 개의 geography 속성을 모두 읽어 toString() 과 getSrid() 를 SQL Server 의 STAsText() 와 STSrid 에 비교한다. 이력 테이블과 뷰의 geography 컬럼 일곱 개는 바이트를 decode 로 읽어 비교한다. 엔티티로 한 주의 경계선을 바꾸고 다른 주의 경계선을 null 로 지운 뒤, SQL Server 가 읽은 바이트가 쓴 바이트와 같은지도 확인한다.

공간 연산은 엔티티로 할 수 없다. Geography 는 좌표를 담을 뿐 계산하지 않는다. 두 점 사이의 거리를 구하는 STDistance 같은 geography 의 메서드는 SQL 에만 있다. 프로젝트의 연습문제 WWI_APPLICATION_A_01 는 한 도시에서 가장 가까운 도시 다섯 개를 SQL 로 찾는다.

geography 는 비교할 수도 없다. geography 컬럼으로 GROUP BY 하면 SQL Server 는 아래의 오류로 쿼리를 거절한다.

geography 로 GROUP BY 할 때의 오류

```
Msg 249, Level 16, State 1, Server 7320492f066b, Line 1
The type "geography" is not comparable. It cannot be used in the GROUP BY clause.
```

Query Language 에서 GROUP BY c 처럼 엔티티로 묶으면, 구현체는 그 엔티티의 컬럼으로 묶는 SQL 을 만들 수 있다. Customer 나 Supplier 처럼 geography 컬럼이 있는 엔티티라면 그 SQL 은 거절된다. 그래서 연습문제의 Query Language 는 엔티티 대신 SQL 과 같이 식별자와 이름으로 묶는다(Exercises).

바이너리

varbinary(max) 컬럼은 넷이다. 사람의 비밀번호의 해시(People.HashedPassword)와 사진(People.Photo), 재고 품목의 사진(StockItems.Photo), 차량의 센서가 보낸 데이터를 압축한 것(VehicleTemperatures.CompressedSensorData)이다. 설치한 데이터베이스에서 값이 있는 것은 HashedPassword 뿐이다. 1,111 명 가운데 157 명의 행에 32 바이트씩 들어 있다. 사진과 압축한 센서 데이터는 모든 행에서 NULL 이다.

MappedPerson 의 hashedPassword 와 photo

```
@Basic(optional = true)
@Column(name = COLUMN_NAME_HASHED_PASSWORD,
        nullable = true,
        insertable = true,
        updatable = true
)
private byte[] hashedPassword;

@Basic(optional = true)
@Column(name = COLUMN_NAME_PHOTO,
        nullable = true,
        insertable = true,
        updatable = true
)
private byte[] photo;
```

byte[] 는 기본 형이므로 @Lob 없이 매핑된다. SQL Server 의 varbinary(max) 는 2 GB 까지 담지만, JDBC 드라이버가 보통의 바이트 배열로 주고받으므로 @Lob 이 필요 없다. 대신 사진이 들어간다면 엔티티를 읽을 때마다 사진도 함께 읽는다. 기본 속성을 늦게 읽는 @Basic(fetch = FetchType.LAZY) 는 구현체가 지켜야 하는 것이 아니라 힌트일 뿐이고, 매핑된 슈퍼클래스는 그런 결정을 하지 않는다([sql-server-mapping-rules]). **사진이 큰 애플리케이션이라면 엔티티 대신 필요한 컬럼만 선택하는 쿼리를 쓴다.**

toString 은 바이트 배열의 내용 대신 길이만 보인다. MappedPerson 의 toString 은 hashedPassword=32 bytes

처럼 쓴다. 비밀번호의 해시를 로그에 남기지 않기 위해서다. geography 인 속성 가운데 점 하나를 담는 location 과 deliveryLocation 은 WKT 를 그대로 보인다. 주와 나라의 border 는 border=… points 처럼 점의 수만 보인다. 나라의 경계선은 18 만 바이트가 넘으므로, 그 WKT 로 로그를 채우지 않는다.

JSON 과 계산 컬럼

nvarchar(max) 에 담은 JSON

WideWorldImporters 는 JSON 문서를 nvarchar(max) 컬럼에 담는다. SQL Server 2016 에는 JSON 을 위한 형(型, type)이 따로 없었고, JSON 을 다루는 함수만 있었기 때문이다.

Table 19. JSON 을 담은 컬럼

컬럼	담는 것
Application.People.CustomFields	직원의 직함, 입사일, 담당 지역, 수수료율, 다른 언어
Application.People.UserPreferences	웹 사이트의 화면 설정
Application.SystemParameters.ApplicationSettings	애플리케이션의 설정
Sales.Invoices.ReturnedDeliveryData	배송 기사가 보낸 배송 진행 기록, 받은 사람과 받은 시각
Warehouse.StockItems.CustomFields	제조국과 태그

ReturnedDeliveryData 에만 CHECK 제약이 있어서, ISJSON 이 참인 문자열만 들어간다. 다른 컬럼은 형으로 보면 그냥 문자열이다.

재고 품목 USB missile launcher (Green) 의 CustomFields 와 Tags 는 아래와 같다.

StockItemID 가 1 인 재고 품목의 CustomFields 와 Tags

```
StockItemID  CustomFields                                                  Tags
-----------  ------------------------------------------------------------  ---------------
          1  { "CountryOfManufacture": "China", "Tags": ["USB Powered"] }  ["USB Powered"]
```

직원인 Kayla Woodcock 의 CustomFields 에는 OtherLanguages, HireDate, Title, PrimarySalesTerritory, CommissionRate 의 다섯 키가 있다. 입사일도 수수료율도 문자열로 들어 있다.

Jakarta Persistence 에도 JSON 을 위한 형은 없다. 매핑된 슈퍼클래스는 이 컬럼들을 `String` 으로 매핑하고, 문서의 내용은 해석하지 않는다.

MappedPerson 의 customFields

```
@Basic(optional = true)
@Column(name = COLUMN_NAME_CUSTOM_FIELDS,
        nullable = true,
        insertable = true,
        updatable = true
)
private String customFields;
```

`nvarchar(max)` 는 길이의 한도가 없는 것과 같으므로 `length` 와 `@Size` 를 두지 않는다([sql-server-types]). 문서를 객체로 다루려면 애플리케이션이 Jakarta JSON Binding 같은 라이브러리로 문자열을 바꾼다. `AttributeConverter` 로 문서를 객체로 바꾸어 매핑할 수도 있지만, 그것은 문서의 형태를 정하는 일이므로 데이터베이스에 정의된 것만 담는 매핑된 슈퍼클래스가 할 일이 아니다.

문서 안의 값으로 쿼리하는 것은 SQL 의 몫이다. SQL Server 의 `JSON_VALUE` 는 JSON 경로로 값 하나를, `JSON_QUERY` 는 객체나 배열을 꺼낸다. Query Language 에는 이런 함수가 없다. 프로젝트의 연습문제 `WWI_APPLICATION_I_02` 는 `CustomFields` 의 `$.Title` 로 직원을 직함마다 세고, 프로젝트의 연습문제 `WWI_WAREHOUSE_I_02` 는 `$.CountryOfManufacture` 로 재고 품목을 제조국마다 센다. 둘 다 SQL 로만 푼다.

계산 컬럼

계산 컬럼(computed column)은 다른 컬럼의 값 등을 사용하는 식으로 값을 계산하는 컬럼이다. WideWorldImporters 에는 계산 컬럼이 여덟 개 있고, 그 가운데 다섯 개가 JSON 에서 값을 꺼낸다.

Table 20. WideWorldImporters 의 계산 컬럼

컬럼	식
People.OtherLanguages	json_query([CustomFields],N'$.OtherLanguages')
People.SearchName	concat([PreferredName],N' ',[FullName])
StockItems.Tags	json_query([CustomFields],N'$.Tags')
StockItems.SearchDetails	concat([StockItemName],N' ',[MarketingComments])
Invoices.ConfirmedDeliveryTime	TRY_CONVERT([datetime2](7),json_value([ReturnedDeliveryData],N'$.DeliveredWhen'),(126))
Invoices.ConfirmedReceivedBy	json_value([ReturnedDeliveryData],N'$.ReceivedBy')
CustomerTransactions.IsFinalized	case when [FinalizationDate] IS NULL then CONVERT([bit],(0)) else CONVERT([bit],(1)) end
SupplierTransactions.IsFinalized	위와 같다.

SearchName 과 두 IsFinalized 는 PERSISTED 여서 계산한 값을 저장해 두고, 나머지 다섯은 읽을 때마다 계산한다. 어느 쪽이든 계산 컬럼에는 값을 쓸 수 없고, SQL Server 는 값을 주는 INSERT 와 UPDATE 를 오류 271 로 거절한다. 그래서 기간 컬럼처럼 읽기 전용으로 매핑한다.

MappedStockItem 의 customFields, tags, searchDetails

```
@Basic(optional = true)
@Column(name = COLUMN_NAME_CUSTOM_FIELDS,
        nullable = true,
        insertable = true,    ❶
        updatable = true
)
private String customFields;

// 읽기 전용: 계산 컬럼이다. SQL Server 가 값을 계산하고, 이 컬럼에 주는 값은 거절한다(오류 271)
@Basic(optional = true)
@Column(name = COLUMN_NAME_TAGS,
        nullable = true,
        insertable = false,   ❷
        updatable = false
)
private String tags;

// 읽기 전용: 계산 컬럼이다. SQL Server 가 값을 계산하고, 이 컬럼에 주는 값은 거절한다(오류 271)
@Basic(optional = false)
@Column(name = COLUMN_NAME_SEARCH_DETAILS,
        nullable = false,
        insertable = false,
        updatable = false
)
private String searchDetails; ❸
```

❶ JSON 문서는 애플리케이션이 쓴다.

❷ 그 문서에서 꺼낸 태그의 배열은 SQL Server 가 계산한다.

❸ NOT NULL 이지만 @NotNull 은 없다.

customFields 를 고치면 다음에 행을 읽을 때 tags 도 바뀐다. 영속성 컨텍스트에 있는 엔티티의 tags 는 고치기 전의 값 그대로이므로, 바뀐 값이 필요하면 엔티티를 다시 읽는다. 시스템 버전 관리 임시 테이블 절의 기간 컬럼과 같다.

searchDetails 에 @NotNull 이 없는 것도 기간 컬럼과 같은 이유다. 새 엔티티를 저장할 때 값은 아직 없고, 행을 넣은 뒤에야 SQL Server 가 계산한다. MappedPerson 의 searchName 도 nullable = false 이고 @Size 는 있지만 @NotNull 은 없다.

JSON 에서 계산한 컬럼은 형도 SQL Server 가 정한다. `ConfirmedDeliveryTime` 은 `TRY_CONVERT` 로 바꾼 `datetime2` 이고, `ConfirmedReceivedBy` 는 `JSON_VALUE` 가 돌려주는 `nvarchar(4000)` 이다. 매핑도 그 형을 따른다.

MappedInvoice 의 returnedDeliveryData, confirmedDeliveryTime, confirmedReceivedBy

```
@Basic(optional = true)
@Column(name = COLUMN_NAME_RETURNED_DELIVERY_DATA,
        nullable = true,
        insertable = true,
        updatable = true
)
private String returnedDeliveryData;

// 읽기 전용: 계산 컬럼이다. SQL Server 가 값을 계산하고, 이 컬럼에 주는 값은 거절한다(오류 271)
@Basic(optional = true)
@Column(name = COLUMN_NAME_CONFIRMED_DELIVERY_TIME,
        nullable = true,
        insertable = false,
        updatable = false
)
private LocalDateTime confirmedDeliveryTime;

// 읽기 전용: 계산 컬럼이다. SQL Server 가 값을 계산하고, 이 컬럼에 주는 값은 거절한다(오류 271)
@Size(max = COLUMN_LENGTH_CONFIRMED_RECEIVED_BY)
@Basic(optional = true)
@Column(name = COLUMN_NAME_CONFIRMED_RECEIVED_BY,
        nullable = true,
        insertable = false,
        updatable = false,
        length = COLUMN_LENGTH_CONFIRMED_RECEIVED_BY
)
private String confirmedReceivedBy;
```

이 두 컬럼 덕분에 JSON 문서 안의 값을 엔티티의 보통 속성으로 읽고, Query Language 로 비교하고 정렬할 수 있다. Jakarta Persistence 의 API 만으로 JSON 의 값을 쿼리하는 방법은, 네이티브 쿼리를 빼면 이것뿐이다. 프로젝트의 연습문제 `WWI_SALES_I_03` 는 고객의 송장을 배송을 확인한 시각과 받은 사람과 함께 읽는다.

`IsFinalized` 는 JSON 과 관계없지만 같은 방식으로 매핑한다. `FinalizationDate` 가 있으면 1, 없으면 0 인 `bit` 이므로 `Boolean` 이고, 읽기 전용이다. 프로젝트의 `WWI_PURCHASING_B_02` 연습문제는 이 컬럼으로 마감되지 않은 공급자 거래를 찾는다.

Exercises

이 장의 데이터로 풀 수 있는 연습문제 가운데 이 장의 매핑을 연습하기 좋은 문제를 선택해 모았다. 문제마다 Query Language 와 Criteria API 로 푼 답을 함께 실었다. 풀이는 프로젝트의 `src/test/sql-exercises` 에 있는 SQL 파일과, 같은 문제를 Query Language 와 Criteria API 로 푼 테스트에 있다.

WWI_PURCHASING_I_01

기간을 받아, 공급자마다 그동안 낸 발주의 수와 예상 단가로 계산한 발주 금액의 합을 금액이 큰 순으로 구한다. 2015 년 한 해로는 두 공급자가 나온다.

SQL

```
DECLARE @from date = '2015-01-01', @to date = '2016-01-01';
SELECT s.SupplierName, COUNT(DISTINCT po.PurchaseOrderID) AS Orders,
       SUM(pol.OrderedOuters * pol.ExpectedUnitPricePerOuter) AS Amount
FROM Purchasing.PurchaseOrders po
JOIN Purchasing.PurchaseOrderLines pol ON pol.PurchaseOrderID = po.PurchaseOrderID
JOIN Purchasing.Suppliers s ON s.SupplierID = po.SupplierID
WHERE po.OrderDate >= @from AND po.OrderDate < @to
GROUP BY s.SupplierID, s.SupplierName
ORDER BY Amount DESC, s.SupplierID;
```

`@from` 과 `@to` 가 2015 년 1 월 1 일과 2016 년 1 월 1 일일 때의 결과는 아래와 같다.

결과

```
SupplierName    Orders        Amount
--------------  ------  ------------
Fabrikam, Inc.     309  295332354.00
Litware, Inc.      301  127101914.10
```

2015 년에 발주를 받은 공급자는 두 곳뿐이다. `Fabrikam, Inc.` 는 옷을, `Litware, Inc.` 는 포장재를 공급한다. 기간은 `@from` 을 넣고 `@to` 를 빼는 반열린 구간으로 기재했다. 날짜에 시각이 붙어도 같은 조건으로 한 해를 정확히 나눈다.

Query Language

```
SELECT s.supplierName, COUNT(DISTINCT po) AS orders,
       SUM(pol.orderedOuters * pol.expectedUnitPricePerOuter) AS amount
FROM PurchasingPurchaseOrderLine AS pol
JOIN pol.purchaseOrder AS po
JOIN po.supplier AS s
WHERE po.orderDate >= :from AND po.orderDate < :to
GROUP BY s.supplierId, s.supplierName
ORDER BY amount DESC, s.supplierId
```

`SUM` 안에 `pol.orderedOuters * pol.expectedUnitPricePerOuter` 처럼 산술식을 쓸 수 있다. `Integer` 와 `BigDecimal` 을 곱하므로 결과는 `BigDecimal` 이다. 한 발주에 품목이 여럿이므로 발주는 `COUNT(DISTINCT po)` 로 센다. 매개변수 `:from` 과 `:to` 에는 `LocalDate` 를 넘긴다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var root = query.from(PurchaseOrderLine.class);
final var order = root.join(PurchaseOrderLine_.purchaseOrder);
final var supplier = order.join(PurchaseOrder_.supplier);
final var amount =
        builder.sum(builder.prod(root.get(PurchaseOrderLine_.orderedOuters),
                                 root.get(
                                         PurchaseOrderLine_.expectedUnitPricePerOuter)));
query.select(builder.array(supplier.get(Supplier_.supplierName),
                           builder.countDistinct(order), amount));
query.where(builder.greaterThanOrEqualTo(order.get(PurchaseOrder_.orderDate),
                                         FROM_2015),
            builder.lessThan(order.get(PurchaseOrder_.orderDate), TO_2015));
query.groupBy(supplier.get(Supplier_.supplierId),
              supplier.get(Supplier_.supplierName));
query.orderBy(builder.desc(amount),
              builder.asc(supplier.get(Supplier_.supplierId)));
return entityManager.createQuery(query).getResultList()
        .stream().map(WWI_PURCHASING_I_01_PersistenceIT::row).toList();
```

곱셈은 `builder.prod(…)`, 중복을 뺀 개수는 `builder.countDistinct(…)` 다. `FROM_2015` 와 `TO_2015` 는 `__Exercises_TestUtils` 의 상수로, `LocalDate.of(2015, 1, 1)` 과 `LocalDate.of(2016, 1, 1)` 이다.

WWI_SALES_I_07

신용 한도가 있는 고객 가운데 한도를 가장 많이 쓴 고객 열 명을, 한도와 마감되지 않은 거래의 남은 금액, 그 금액이 한도에서 차지하는 백분율과 함께 구한다. 같으면 `CustomerID` 순이다.

고객마다 남은 금액과 그 백분율을 `SELECT` 절의 스칼라 서브쿼리로 구한다. 고객의 엔티티 `c` 를 선택하면서 `GROUP BY c` 로 묶으면 `Sales.Customers` 의 컬럼을 모두 묶게 되는데, SQL Server 는 그 가운데 `geography` 컬럼으로 묶지 못한다. 그래서 고객으로 묶지 않고, 서브쿼리가 고객마다 마감되지 않은 거래의 `outstandingBalance` 를 더한다.

Query Language

```
SELECT c,
       (SELECT SUM(ct.outstandingBalance)
        FROM SalesCustomerTransaction ct
        WHERE ct.customer = c
          AND ct.isFinalized = false) AS outstanding,
       (SELECT SUM(ct.outstandingBalance) * 100 / c.creditLimit
        FROM SalesCustomerTransaction ct
        WHERE ct.customer = c
          AND ct.isFinalized = false) AS percentUsed
FROM SalesCustomer c
WHERE c.creditLimit > 0
ORDER BY percentUsed DESC NULLS LAST, c.customerId
```

백분율의 서브쿼리는 남은 금액을 다시 더한다. `SELECT` 절의 별칭 `outstanding` 은 다른 선택 항목에서 쓸 수 없기 때문이다. 마감되지 않은 거래가 없는 고객은 서브쿼리가 `NULL` 을 돌려주므로, `NULLS LAST` 로 그런 고객을 뒤로 보낸다. `ORDER BY` 의 `NULLS FIRST` 와 `NULLS LAST` 는 Jakarta Persistence 3.2 에서 Query Language 에 추가되었다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var c = query.from(Customer.class);
final var outstanding = query.subquery(BigDecimal.class);
final var ct = outstanding.from(CustomerTransaction.class);
outstanding.select(builder.sum(ct.get(CustomerTransaction_.outstandingBalance)))
        .where(builder.equal(ct.get(CustomerTransaction_.customer), c),
               builder.isFalse(ct.get(CustomerTransaction_.isFinalized)));
final var creditLimit = c.get(Customer_.creditLimit);
final var percentUsed = builder.quot(builder.prod(outstanding, BigDecimal.valueOf(100)),
                                     creditLimit);
query.select(builder.array(c, outstanding, percentUsed))
        .where(builder.gt(creditLimit, BigDecimal.ZERO))
        .orderBy(builder.desc(percentUsed, jakarta.persistence.criteria.Nulls.LAST),
                 builder.asc(c.get(Customer_.customerId)));
return entityManager.createQuery(query).setMaxResults(10).getResultList();
```

`jakarta.persistence.criteria.Nulls` 의 상수는 아래와 같다. 3.2 에서 새로 생긴 열거형이다.

Table 21. jakarta.persistence.criteria.Nulls 의 상수 (3.2)

상수	뜻
`NONE`	`NULL` 의 순서를 정하지 않는다.
`FIRST`	`NULL` 이 결과의 앞에 온다.
`LAST`	`NULL` 이 결과의 끝에 온다.

Criteria API 에서는 서브쿼리 `outstanding` 을 한 번 생성하고, 선택 항목과 백분율의 식에 함께 쓴다. `builder.desc(percentUsed, Nulls.LAST)` 가 `DESC NULLS LAST` 이고, `Nulls` 와 이 메서드도 Jakarta Persistence 3.2 에서 추가되었다.

WWI_WAREHOUSE_I_01

재고 그룹마다 재고 품목의 수를 많은 순으로 구한다. 품목이 없는 그룹도 나와야 하고, 같으면 그룹의 이름순이다.

SQL

```
SELECT sg.StockGroupName, COUNT(sisg.StockItemStockGroupID) AS StockItems
FROM Warehouse.StockGroups sg
LEFT JOIN Warehouse.StockItemStockGroups sisg ON sisg.StockGroupID = sg.StockGroupID
GROUP BY sg.StockGroupID, sg.StockGroupName
ORDER BY StockItems DESC, sg.StockGroupName;
```

결과

```
StockGroupName       StockItems
-------------------  ----------
Novelty Items                91
Computing Novelties          83
Clothing                     74
Packaging Materials          67
Mugs                         42
T-Shirts                     26
Furry Footwear               24
Toys                         21
USB Novelties                14
Airline Novelties             0
```

합하면 442 개로 재고 품목 227 개보다 많다. `StockItemStockGroups` 는 품목과 그룹을 잇는 다대다 테이블이어서, 한 품목이 여러 그룹에 든다. `Airline Novelties` 에는 품목이 하나도 없다. 그룹에서 다대다 테이블로 `LEFT JOIN` 하므로 이 그룹도 0 개로 나온다.

Query Language

```
SELECT sg.stockGroupName, COUNT(sisg) AS stockItems
FROM WarehouseStockGroup AS sg
LEFT JOIN WarehouseStockItemStockGroup AS sisg ON sisg.stockGroup = sg
GROUP BY sg.stockGroupId, sg.stockGroupName
ORDER BY stockItems DESC, sg.stockGroupName
```

`StockItemStockGroup` 은 다대다 테이블을 매핑한 엔티티다. 이 프로젝트는 `@ManyToMany` 를 쓰지 않고, 다대다 테이블도 `@ManyToOne` 두 개를 가진 엔티티로 매핑한다. `StockGroup` 에는 이 엔티티로 가는 관계가 없으므로, 엔티티를 `LEFT JOIN` 하고 `ON sisg.stockGroup = sg` 로 조인 조건을 기재한다. `COUNT(sisg)` 는 SQL 의 `COUNT(sisg.StockItemStockGroupID)` 처럼 품목이 없는 그룹을 0 으로 센다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var group = query.from(StockGroup.class);
// StockGroup 에는 StockItemStockGroup 으로 가는 관계가 없으므로 ON 으로 엔티티 조인한다
final var root = group.join(StockItemStockGroup.class, JoinType.LEFT);
root.on(builder.equal(root.get(StockItemStockGroup_.stockGroup), group));
final var name = group.get(StockGroup_.stockGroupName);
final var stockItems = builder.count(root);
query.select(builder.array(name, stockItems));
query.groupBy(group.get(StockGroup_.stockGroupId), name);
query.orderBy(builder.desc(stockItems), builder.asc(name));
return entityManager.createQuery(query).getResultList()
        .stream().map(WWI_WAREHOUSE_I_01_PersistenceIT::row).toList();
```

`group.join(StockItemStockGroup.class, JoinType.LEFT)` 는 관계가 아니라 엔티티의 클래스로 조인하는, Jakarta Persistence 3.2 에서 추가된 메서드이고, `on` 에 넘긴 조건이 `ON` 이 된다. 식별자와 이름으로 묶지만 선택하는 것은 이름과 개수다.

WWI_WAREHOUSE_I_03

재고 품목과 한 해를 받아, 그해 그 품목의 입출고를 거래의 종류마다 거래의 수와 수량의 합으로, 종류의 이름순으로 구한다. 품목 1 의 2015 년으로는 `Stock Issue` 한 종류만 나온다.

다대일 관계로 조인한 거래의 종류 `tt` 를 엔티티째 선택하고, `GROUP BY tt` 로 그 엔티티로 묶는다. 엔티티로 묶으면 구현체가 그 엔티티의 컬럼으로 묶는 SQL 을 만든다. 기간은 `:from` 을 넣고 `:to` 를 빼는 반열린 구간으로 기재한다.

Query Language

```
SELECT tt, COUNT(sit) AS transactions, SUM(sit.quantity) AS quantity
FROM WarehouseStockItemTransaction sit JOIN sit.transactionType tt
WHERE sit.stockItemId = :stockItemId
  AND sit.transactionOccurredWhen >= :from
  AND sit.transactionOccurredWhen < :to
GROUP BY tt
ORDER BY tt.transactionTypeName
```

`transactionOccurredWhen` 은 `LocalDateTime` 이므로, `:from` 과 `:to` 에는 `FROM_2015` 와 `TO_2015` 에 `atStartOfDay()` 를 호출한 `LocalDateTime` 을 넘긴다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var sit = query.from(StockItemTransaction.class);
final var tt = sit.join(StockItemTransaction_.transactionType);
final var occurredWhen = sit.get(StockItemTransaction_.transactionOccurredWhen);
query.select(builder.array(tt, builder.count(sit),
                           builder.sum(sit.get(StockItemTransaction_.quantity))))
        .where(builder.equal(sit.get(StockItemTransaction_.stockItemId), STOCK_ITEM_ID),
               builder.greaterThanOrEqualTo(occurredWhen, FROM),
               builder.lessThan(occurredWhen, TO))
        .groupBy(tt)
        .orderBy(builder.asc(tt.get(TransactionType_.transactionTypeName)));
return entityManager.createQuery(query).getResultList();
```

`groupBy(tt)` 처럼 조인을 그대로 넘기면 엔티티로 묶는다. `select`, `where`, `groupBy`, `orderBy` 는 쿼리 자신을 돌려주므로 이어서 호출할 수 있다.

WWI_WAREHOUSE_I_04

기간을 받아, 그동안 입출고가 있는 재고 품목마다 들어온 수량, 양수로 나타낸 나간 수량, 순 변동량을 품목의 이름순으로 구한다. 2016 년 5 월 한 달로는 재고 품목 227 개가 모두 나온다.

거래의 `quantity` 는 들어오면 양수, 나가면 음수다. `SUM` 안의 `CASE` 식이 들어온 수량과 나간 수량을 나누므로, 한 번의 조회로 세 합을 함께 구한다.

Query Language

```
SELECT si,
       SUM(CASE WHEN st.quantity > 0 THEN st.quantity ELSE 0 END)
           AS incoming,
       SUM(CASE WHEN st.quantity < 0 THEN -st.quantity ELSE 0 END)
           AS outgoing,
       SUM(st.quantity) AS netMovement
FROM WarehouseStockItemTransaction st
         JOIN st.stockItem si
WHERE st.transactionOccurredWhen >= :from
  AND st.transactionOccurredWhen < :to
GROUP BY si
ORDER BY si.stockItemName
```

나간 수량은 `-st.quantity` 처럼 단항 연산자 `-` 로 양수로 바꾼다. `SUM(st.quantity)` 는 들어온 수량에서 나간 수량을 뺀 순 변동량이다.

Criteria API

```
final var builder = entityManager.getCriteriaBuilder();
final var query = builder.createQuery(Object[].class);
final var st = query.from(StockItemTransaction.class);
final var si = st.join(StockItemTransaction_.stockItem);
final var quantity = st.get(StockItemTransaction_.quantity);
final var occurredWhen = st.get(StockItemTransaction_.transactionOccurredWhen);
final var incoming = builder.<BigDecimal>selectCase()
        .when(builder.gt(quantity, BigDecimal.ZERO), quantity)
        .otherwise(BigDecimal.ZERO);
final var outgoing = builder.<BigDecimal>selectCase()
        .when(builder.lt(quantity, BigDecimal.ZERO), builder.neg(quantity))
        .otherwise(BigDecimal.ZERO);
query.select(builder.array(si, builder.sum(incoming), builder.sum(outgoing),
                           builder.sum(quantity)))
        .where(builder.greaterThanOrEqualTo(occurredWhen, FROM),
               builder.lessThan(occurredWhen, TO))
        .groupBy(si)
        .orderBy(builder.asc(si.get(StockItem_.stockItemName)));
return entityManager.createQuery(query).getResultList();
```

`CASE` 식은 `builder.selectCase()` 로 생성하고, `when` 과 `otherwise` 를 이어서 호출한다. `selectCase()` 는 인자가 없어 결과의 형을 알 수 없으므로 `builder.<BigDecimal>selectCase()` 처럼 형 인자를 기재한다. 단항 연산자 `-` 는 `builder.neg(…)` 다.

[1] Microsoft. Wide World Importers - Sample Database for SQL. "Wide World Importers (WWI) is a wholesale novelty goods importer and distributor operating from the San Francisco Bay area." https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-what-is

[2] Microsoft. WideWorldImporters OLTP database catalog. https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-oltp-database-catalog

[3] Microsoft. Temporal Tables. "The system uses the history table to automatically store the previous version of the row each time a row in the temporal table gets updated or deleted." https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal/overview

[4] Microsoft. Temporal Tables. "FOR SYSTEM_TIME filters out rows that have a period of validity with zero duration (ValidFrom = ValidTo)." https://learn.microsoft.com/en-us/sql/relational-databases/tables/temporal/overview

[5] Microsoft. WideWorldImporters OLTP database catalog. "Auto-numbering in tables is based on sequences. These sequences are easier to work with across linked servers and similar environments than IDENTITY columns." https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-oltp-database-catalog

[6] Microsoft. WideWorldImporters OLTP database catalog. "Memory-optimized tables use IDENTITY columns since they don't support in SQL Server 2016." https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-oltp-database-catalog

[7] Microsoft. WideWorldImporters OLTP database catalog. "Non-temporal columns for stock items. These are frequently updated columns." https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-oltp-database-catalog

[8] Microsoft. WideWorldImporters OLTP database catalog. "All access to the database from the company website is through this schema." https://learn.microsoft.com/en-us/sql/samples/wide-world-importers-oltp-database-catalog

[9] Microsoft. [MS-SSCLRT]: Microsoft SQL Server CLR Types Serialization Formats. https://learn.microsoft.com/en-us/openspecs/sql_server_protocols/ms-ssclrt/